

Approximate Set Cover Problem via Randomization

Algorithm Engineering - Prof. Mattia D'Emidio

Autore: Nino Giuseppe Critelli

Anno Accademico 2025/2026
Università degli Studi dell'Aquila
DISIM - LM Ing. Informatica

Indice

1	Introduzione	3
2	Randomizzazione	4
3	Generazione delle istanze	7
4	Algoritmi	9
4.1	Brute Force	9
4.2	ILP — Integer Linear Programming	9
4.3	Greedy deterministico	10
4.4	Greedy con pattern Hash Map	11
4.5	Randomized Rounding	13
5	Strumenti e metodologia	17
6	Pilot	19
6.1	Implementazione degli algoritmi	19
6.2	Test di validazione	19
6.3	Esperimenti	22
6.3.1	Esperimento 1 — Qualità della soluzione	22
6.3.2	Esperimento 2 — Runtime: Greedy vs Randomized Rounding	24
6.3.3	Esperimento 3 — Greedy Worst Case	25
6.3.4	Esperimento 4 — Variabilità della soluzione	26
6.3.5	Esperimento 5 — Effetto del parametro c nel Rand. Round.	27
6.3.6	Esperimento 6 — Tasso di fallimento empirico vs bound teorico	28
6.3.7	Esperimento 7 — Distribuzione di $ \mathcal{C} $ al variare di n	30
6.3.8	Esperimento 8 — Speedup del pattern Hash Map	31
6.3.9	Esperimento 9 — Las Vegas vs Monte Carlo	32
6.4	Riepilogo del Pilot	33
7	Workhorse	35
7.1	Implementazione	35
7.2	Parametri	35
7.3	Esperimenti	35
7.3.1	Esperimento W1 — Runtime e Speedup	35
7.3.2	Esperimento W2 — Qualità della soluzione	37
7.3.3	Esperimento W3 — Doubling Experiment	38
7.3.4	Esperimento W4 — Bottleneck Randomized Rounding	40
8	Conclusioni	42
9	Fonti e utilizzo di LLM	44

1 Introduzione

Il **Set Cover** è uno dei problemi di ottimizzazione combinatoria più studiati.

Consiste nel trovare la copertura (cover) minima di una famiglia di sottoinsiemi di elementi in modo da coprire ogni elemento dell'universo.

Definizione del problema

Dato un universo X con $|X| = n$ elementi e una famiglia $\mathcal{F} = \{S_1, \dots, S_m\}$ di m sottoinsiemi di X , l'obiettivo è trovare una sottofamiglia $\mathcal{C} \subseteq \mathcal{F}$ di dimensione **minima** tale che:

$$\bigcup_{S \in \mathcal{C}} S = X$$

Il problema è **NP-hard**, quindi non è noto alcun algoritmo esatto che lo risolva in tempo polinomiale (assumendo $P \neq NP$). Questo motiva lo studio di algoritmi di approssimazione — algoritmi polinomiali che producono soluzioni la cui qualità è garantita essere entro un fattore ρ dall'ottimo.

Obiettivi del progetto

In questo progetto studieremo alcune tecniche di randomizzazione per l'approssimazione del Problema confrontandole in modo sperimentale con altri algoritmi deterministici. L'obiettivo è dimostrare come la randomizzazione, introducendo casualità, possa migliorare le prestazioni medie pur rispettando le garanzie teoriche nel caso peggiore atteso.

Gli algoritmi trattati nel progetto sono:

- **Brute force** - algoritmo esatto
- **Formulazione ILP - Gurobi solver** - algoritmo esatto
- **Greedy deterministico** — con garanzia $\rho = H(\max |S|) \leq \ln n + 1$
- **Greedy con pattern Hash Map** — stessa garanzia del precedente, complessità migliorata
- **Randomized Rounding - tipo Monte Carlo** — basato su rilassamento LP + rounding, “scommette” sulla soluzione, tempi di risoluzione deterministici.
- **Randomized Rounding - tipo Las Vegas** — basato su rilassamento LP + rounding, “scommette” sul tempo, trova sempre una soluzione

La metodologia seguita è quella dell'*Algorithm Engineering*: studio teorico degli algoritmi, implementazione in Python (pilot) e C++ (workhorse) e valutazione sperimentale sistematica su istanze generate sinteticamente.

2 Randomizzazione

Un **algoritmo randomizzato** è un algoritmo che introduce deliberatamente casualità nel processo di calcolo, tramite una procedura $\text{RANDOM}(\mathbf{a}, \mathbf{b})$ che restituisce un intero casuale in $\{a, \dots, b\}$. A differenza degli algoritmi deterministici, il comportamento dipende non solo dall'input x ma anche da una sequenza di scelte casuali R .

Di seguito, alcuni concetti teorici che saranno valutati in questo progetto:

Running time come variabile aleatoria

Per un algoritmo randomizzato, fissato l'input x , il tempo di esecuzione dipende dalle scelte casuali R e si denota $T(x; R)$. Esecuzioni diverse sullo stesso input possono produrre tempi diversi. Il tempo di esecuzione è quindi una **variabile aleatoria**.

Si distinguono due garanzie complementari:

- **Tempo atteso** — fissato un input x , si calcola la media sul comportamento casuale dell'algoritmo:

$$T_{\text{exp}}(n) = \max_{|x|=n} \mathbb{E}_R[T(x; R)]$$

- **Tempo worst-case** — l'avversario sceglie sia l'input x che la sequenza sfortunata R :

$$T_{wc}(n) = \max_{x, R} T(x; R)$$

Valore atteso e linearità

Il **valore atteso** di una variabile aleatoria discreta X con esiti x_i di probabilità p_i è:

$$\mathbb{E}[X] = \sum_i p_i x_i$$

Una proprietà fondamentale è la **linearità dell'aspettazione**: per qualsiasi insieme di variabili aleatorie $X_1, \dots, X_m$ (anche dipendenti):

$$\mathbb{E}\left[\sum_{i=1}^m X_i\right] = \sum_{i=1}^m \mathbb{E}[X_i]$$

Variabile casuale indicatrice

Data un evento casuale A , la sua **variabile indicatrice** è:

$$I_A = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Vale il lemma fondamentale $\mathbb{E}[I_A] = \Pr[A]$, perché:

$$\mathbb{E}[I_A] = 1 \cdot \Pr[A] + 0 \cdot \Pr[\neg A] = \Pr[A]$$

Le variabili indicatrici convertono probabilità in aspettative e, combinate con la linearità, semplificano l'analisi di algoritmi randomizzati.

With High Probability (WHP)

Si dice che una sequenza di eventi (E_n) accade **con alta probabilità** (w.h.p.) se:

$$\Pr[E_n] \geq 1 - \frac{1}{\text{poly}(n)}$$

L'intuizione è che per n sufficientemente grande la proprietà desiderata è praticamente certa. In questo progetto, il Randomized Rounding garantisce $\Pr[\text{successo}] \geq 1 - n^{1-c}$ — una garanzia w.h.p. per $c \geq 2$.

Pro e contro degli algoritmi randomizzati

Pro: codice più semplice; prestazioni medie migliori; robustezza agli input avversariali; buone soluzioni approssimate.

Contro: il comportamento è stocastico; le garanzie sono in aspettazione (in media) o valgono solo con alta probabilità -> perdita di certezza assoluta.

Classificazione: Monte Carlo vs Las Vegas

- **Monte Carlo** — il tempo di esecuzione è deterministico, la soluzione può essere scorretta con bassa probabilità (controllabile). L'errore è rilevabile se l'algoritmo è *one-sided*: una soluzione errata può essere identificata e scartata (come nel Randomized Rounding, dove la validità della cover si verifica in $O(n)$).
- **Las Vegas** — la soluzione è sempre corretta, il tempo di esecuzione è una variabile aleatoria. L'algoritmo termina con probabilità 1. Un algoritmo Las Vegas è **efficiente** se il suo tempo atteso è polinomiale nella dimensione dell'input. È particolarmente utile quando il worst-case deterministico è causato da input *corner case* rari — la randomizzazione li evita pur mantenendo la semplicità del codice. Lo svantaggio principale è che è spesso difficile da analizzare formalmente. Nel nostro progetto, il Randomized Rounding Las Vegas risolve il LP una volta sola e ripete il rounding finché la cover è valida: correttezza garantita, ma tempo atteso che cresce rapidamente con n .

Pattern di randomizzazione

Si possono identificare tre pattern principali di randomizzazione:

1. **Best-from-Random-Order** — presentare l'input in ordine casuale e selezionare la migliore soluzione trovata. Non applicabile al greedy Set Cover perché l'algoritmo è *order-insensitive*: sceglie sempre l'insieme con guadagno massimo indipendentemente dall'ordine di presentazione.
2. **Reduce-to-Hash-Tables** — sostituire una ricerca lineare $O(n)$ con un'operazione su hash table a tempo $O(1)$ atteso. Applicato al greedy Set Cover tramite una mappa inversa - elemento $\rightarrow$ insiemi contenenti quell'elemento - e un Max-Heap con lazy deletion, riducendo la complessità da $O(n \cdot m)$ a $O(\sum_j |S_j| \cdot \log m)$.

3. **Randomized Rounding** — rilassare un problema ILP a LP, risolverlo in tempo polinomiale ottenendo una soluzione frazionaria $\hat{x}$, e arrotondare stocasticamente: include S_j con probabilità $\hat{x}_j$. Produce un algoritmo Monte Carlo con garanzia $\mathbb{E}[|\mathcal{C}|] \leq \lceil c \ln n \rceil \cdot \text{OPT}$ e probabilità di fallimento $\Pr[\text{fallisce}] \leq n^{1-c}$.

3 Generazione delle istanze

Le istanze (universo e sottoinsiemi degli elementi) per gli esperimenti di questo progetto sono sintetici (no data set reali) e seguono il modello di probabilità di **Bernoulli**.

Modello Bernoulli

Dato un universo $X = \{1, \dots, n\}$ e una famiglia di m insiemi, ogni elemento $e \in X$ viene incluso nell'insieme S_j con probabilità indipendente $p \in (0, 1]$:

$$\Pr[e \in S_j] = p \quad \forall e \in X, \forall j \in \{1, \dots, m\}$$

Garanzia di copertura

Per garantire che ogni istanza generata ammetta una soluzione valida, dopo la generazione si verifica che ogni elemento $e \in X$ sia contenuto in almeno un insieme. Se un elemento non è coperto, viene inserito in un insieme scelto casualmente. Questo assicura che il generatore produca solo istanze **ammissibili**.

Algorithm 1: Generatore istanze - pseudocodice

Input: $n, m \in \mathbb{N}, p \in (0, 1]$
Output: universo X , famiglia $\mathcal{F} = \{S_1, \dots, S_m\}$

```
1  $X \leftarrow \{1, \dots, n\};$ 
2 Inizializza  $S_j \leftarrow \emptyset$  per ogni  $j = 1, \dots, m;$ 
3 for  $j = 1$  to  $m$  do
4   for  $e = 1$  to  $n$  do
5     if  $\text{RANDOM}(0, 1) < p$  then
6        $S_j \leftarrow S_j \cup \{e\};$ 
// Garanzia di copertura
7 for  $e = 1$  to  $n$  do
8   if  $e \notin \bigcup_j S_j$  then
9     scegli  $j^* \leftarrow \text{RANDOM}(1, m);$ 
10     $S_{j^*} \leftarrow S_{j^*} \cup \{e\};$ 
11 return  $X, \mathcal{F};$ 
```

La dimensione attesa di ogni insieme è $\mathbb{E}[|S_j|] = n \cdot p$.

Il seed garantisce la riproducibilità dell'esperimento: per uno stesso seed, la funzione genera uguale universo e uguale famiglia di sottoinsiemi

La scelta di p influenza significativamente la struttura delle istanze: valori piccoli ($p = 0.1$) producono insiemi **sparsi**, valori grandi ($p = 0.5$) producono insiemi **densi**.

Test sui generatori di istanze

In test/utils.py sono implementati alcuni test di validazione e di studio. Il generatore è stato validato su quattro proprietà fondamentali:

1. **TEST 1: Copertura garantita** — anche con $p = 0.0$, ogni elemento è presente in almeno un insieme grazie al meccanismo di garanzia.

2. **TEST 2: Effetto di p sulla densità** — la dimensione media degli insiemi cresce con p , come atteso dal modello teorico $\mathbb{E}[|S_j|] = n \cdot p$:

Tabella 1: `utils.py – testIstanceGenerators()` – $n = 10$, $m = 4$, `seed=123`

p	media $ S_j $	intervallo
0.1	3.0	[3, 3]
0.3	4.0	[3, 5]
0.5	6.0	[4, 9]
0.7	7.5	[5, 10]
0.9	9.2	[8, 10]

3. **TEST 3: Effetto del seed sulla variabilità** — seed diversi producono istanze diverse con distribuzioni variabili, confermando che il generatore non è degenere.
4. **TEST 4: Riproducibilità** — lo stesso seed produce sempre la stessa istanza, garantendo la replicabilità degli esperimenti.

4 Algoritmi

Di seguito si descrivono gli algoritmi implementati in questo progetto. Per ciascuno, viene presentato:

- pseudocodice
- complessità e garanzie di approssimazione teoriche
- risultato dei test di validità eseguiti

4.1 Brute Force

L'algoritmo **Brute Force** risolve il problema Set Cover in modo **esatto** enumerando tutti i 2^m sottoinsiemi della famiglia $\mathcal{F}$ e restituendo quello di dimensione minima che copre l'universo X .

Algorithm 2: Brute Force Set Cover - pseudocodice

Input: universo X , famiglia $\mathcal{F} = \{S_1, \dots, S_m\}$

Output: cover ottima $\mathcal{C}^*$

```
1  $\mathcal{C}^* \leftarrow \mathcal{F}$ ;  
2 for ogni sottoinsieme  $\mathcal{C} \subseteq \mathcal{F}$  do  
3   if  $\bigcup_{S \in \mathcal{C}} S = X$  and  $|\mathcal{C}| < |\mathcal{C}^*|$  then  
4      $\mathcal{C}^* \leftarrow \mathcal{C}$ ;  
5 return  $\mathcal{C}^*$ ;
```

Complessità: $O(2^m \cdot n)$ — esponenziale in m . Utilizzato esclusivamente come riferimento per calcolare OPT su istanze piccole ($m \leq 20$, $n \leq 20$).

Test di validazione

La correttezza del Brute Force è verificata in `test/algoritmi/brute_force.py` tramite la funzione `testBruteForce()`. I test coprono quattro casi:

1. **TEST 1** — istanza semplice con $|X| = 5$, $m = 4$: cover ottima trovata $\mathcal{C}^* = \{S_0, S_2\}$, $|\mathcal{C}^*| = 2$.
2. **TEST 2** — istanza con un insieme che copre tutto ($|X| = 3$, $m = 3$): cover ottima $\mathcal{C}^* = \{S_0\}$, $|\mathcal{C}^*| = 1$.
3. **TEST 3** — istanza generata casualmente ($n = 10$, $m = 5$, $p = 0.4$, `seed=42`): cover valida trovata con $|\mathcal{C}^*| = 3$.
4. **TEST 4** — istanza worst-case per Greedy ($k = 4$, $n = 30$): cover ottima $\mathcal{C}^* = \{S_4, S_5\}$, $|\mathcal{C}^*| = 2$.

4.2 ILP — Integer Linear Programming

La formulazione **ILP** (Integer Linear Programming) del problema Set Cover introduce una variabile binaria $x_j \in \{0, 1\}$ per ogni insieme S_j , dove $x_j = 1$ indica che S_j è incluso nella cover:

$$\begin{aligned}
& \min \sum_{j=1}^m x_j \\
& \text{s.t.} \quad \sum_{j: e \in S_j} x_j \geq 1 \quad \forall e \in X \\
& x_j \in \{0, 1\} \quad \forall j \in \{1, \dots, m\}
\end{aligned}$$

Il vincolo garantisce che ogni elemento $e \in X$ sia coperto da almeno un insieme. La soluzione ottima dell'ILP coincide con OPT del problema Set Cover.

Complessità: NP-hard - il problema ILP è equivalente al problema originale. Risolto nel progetto tramite il solver **Gurobi** (Python, fase pilot). Utilizzato come riferimento esatto su istanze medie ($n \leq 20$, $m \leq 50$).

Test di validazione

La correttezza dell'ILP è verificata in `test/algoritmi/ilp.py` tramite la funzione `testILP()`. I test coprono tre casi:

1. **TEST 1** — istanza semplice con $|X| = 5$, $m = 4$: cover ottima $\mathcal{C}^* = \{S_1, S_3\}$, $|\mathcal{C}^*| = 2$.
2. **TEST 2** — istanza generata casualmente ($n = 10$, $m = 5$, $p = 0.4$, `seed=42`): cover valida con $|\mathcal{C}^*| = 3$.
3. **TEST 3** — istanza worst-case per Greedy ($k = 4$, $n = 30$): cover ottima $\mathcal{C}^* = \{S_4, S_5\}$, $|\mathcal{C}^*| = 2$.

4.3 Greedy deterministico

L'algoritmo **Greedy** (ingordo) costruisce la cover iterativamente: ad ogni passo seleziona l'insieme S^* che copre il maggior numero di elementi ancora scoperti (*massimo guadagno*), fino a coprire tutto l'universo X .

Algorithm 3: Greedy Set Cover - pseudocodice

Input: universo X , famiglia $\mathcal{F} = \{S_1, \dots, S_m\}$

Output: cover $\mathcal{C}$

```

1  $\mathcal{C} \leftarrow \emptyset$ ;
2  $U \leftarrow X$  // elementi non ancora coperti
3 while  $U \neq \emptyset$  do
4    $S^* \leftarrow \arg \max_{S \in \mathcal{F}} |S \cap U|$ ;
5    $\mathcal{C} \leftarrow \mathcal{C} \cup \{S^*\}$ ;
6    $U \leftarrow U \setminus S^*$ ;
7 return  $\mathcal{C}$ ;
```

Complessità: $O(n \cdot m)$ per iterazione, $O(\min(m, n) \cdot n \cdot m)$ totale.

Garanzia di approssimazione: il greedy produce una cover di dimensione al più $H(\max_j |S_j|) \cdot \text{OPT}$, dove $H(k) = \sum_{i=1}^k \frac{1}{i}$ è il k -esimo numero armonico. Poiché $H(k) \leq \ln k + 1$, si ha:

$$|\mathcal{C}| \leq (\ln n + 1) \cdot \text{OPT}$$

Questa garanzia è **tight**: esiste una famiglia di istanze (worst-case) per cui il greedy raggiunge esattamente $\rho = k/2$ con $k = \log_2 n$.

Test di validazione

La correttezza del Greedy è verificata in `test/algoritmi/greedy.py` tramite la funzione `testGreedy()`. I test coprono cinque casi:

1. **TEST 1** — istanza semplice con $|X| = 5$, $m = 4$: cover trovata $\mathcal{C} = \{S_0, S_2\}$, $|\mathcal{C}| = 2 = \text{OPT}$.
2. **TEST 2** — istanza con un insieme che copre tutto ($|X| = 3$, $m = 3$): cover $\mathcal{C} = \{S_0\}$, $|\mathcal{C}| = 1 = \text{OPT}$.
3. **TEST 3** — istanza generata casualmente ($n = 10$, $m = 5$, $p = 0.4$, `seed=42`): cover valida trovata.
4. **TEST 4** — istanza generata casualmente ($n = 20$, $m = 8$, $p = 0.5$, `seed=42`): cover con $|\mathcal{C}| = 3$, rispetta il bound $H(14) \cdot \text{OPT} = 3.25 \cdot 3 = 9.75$.
5. **TEST 5** — istanza worst-case ($k = 4$, $n = 30$):

L'universo è $X = \{1, \dots, 30\}$ e la famiglia di insiemi è:

- $S_0 = \{1, 2\}$
- $S_1 = \{3, 4, 5, 6\}$
- $S_2 = \{7, 8, 9, 10, 11, 12, 13, 14\}$
- $S_3 = \{15, 16, \dots, 30\}$
- $S_4 = \{1, 3, 4, 7, 8, 9, 10, 15, 16, 17, 18, 19, 20, 21, 22\}$
- $S_5 = \{2, 5, 6, 11, 12, 13, 14, 23, 24, 25, 26, 27, 28, 29, 30\}$

La soluzione ottima è $\mathcal{C}^* = \{S_4, S_5\}$ con $|\mathcal{C}^*| = 2$, mentre il greedy seleziona $\mathcal{C} = \{S_3, S_2, S_1, S_0\}$ con $|\mathcal{C}| = 4$, scegliendo ad ogni passo l'insieme con guadagno locale massimo senza individuare la soluzione globale ottima. Rapporto empirico $\rho = 2.0$, bound teorico $H(16) \cdot \text{OPT} = 6.76$.

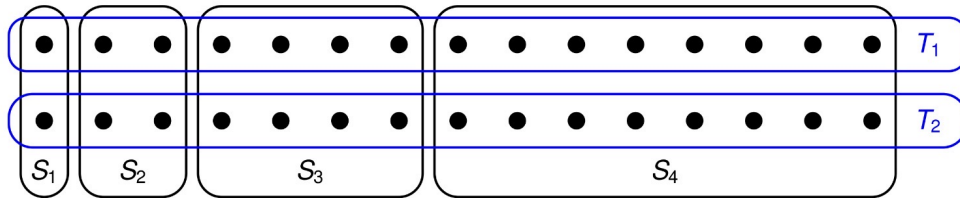


Figura 1: Worst Case Greedy Alg.

4.4 Greedy con pattern Hash Map

Il **Greedy Hash Map** è una versione ottimizzata del greedy deterministico che applica il pattern *Reduce-to-Hash-Tables* per ridurre la complessità.

Idea chiave

Il greedy naive ad ogni iterazione scansiona tutti gli m insiemi per calcolare il guadagno corrente — $O(n \cdot m)$ per iterazione. Il pattern Hash Map sostituisce questa scansione con due strutture dati:

- **Mappa inversa elemento $\rightarrow$ insiemi**: per ogni elemento $e \in X$, mantiene la lista degli insiemi che lo contengono. Permette di aggiornare solo gli insiemi coinvolti in $O(1)$ per lookup.
- **Max-Heap con lazy deletion**: mantiene gli insiemi ordinati per guadagno corrente. Quando il guadagno di un insieme cambia, non viene aggiornato immediatamente nell'heap — il vecchio valore rimane finché non viene estratto (*lazy deletion*).

Algorithm 4: Greedy Set Cover con Hash Map - pseudocodice

Input: universo X , famiglia $\mathcal{F} = \{S_1, \dots, S_m\}$

Output: cover $\mathcal{C}$

```
1 Costruisci mappa inversa:  $\text{elem2sets}[e] \leftarrow \{j : e \in S_j\}$ ;
2  $\text{gain}[j] \leftarrow |S_j|$  per ogni  $j$ ;
3  $H \leftarrow \text{MaxHeap}(\{(\text{gain}[j], j)\})$ ;
4  $\mathcal{C} \leftarrow \emptyset$ ,  $\text{covered} \leftarrow \emptyset$ ;
5 while  $\text{covered} \neq X$  do
6    $(g, j^*) \leftarrow H.\text{extractMax}()$ ;
7   if  $g \neq \text{gain}[j^*]$  then
8      $H.\text{insert}((\text{gain}[j^*], j^*))$  // lazy deletion
9     continue;
10  if  $\text{gain}[j^*] = 0$  then
11    break;
12   $\mathcal{C} \leftarrow \mathcal{C} \cup \{j^*\}$ ;
13  for  $e \in S_{j^*} \setminus \text{covered}$  do
14     $\text{covered} \leftarrow \text{covered} \cup \{e\}$ ;
15    for  $j \in \text{elem2sets}[e]$ ,  $j \neq j^*$  do
16       $\text{gain}[j] \leftarrow \text{gain}[j] - 1$ ;
17 return  $\mathcal{C}$ ;
```

Complessità: $O\left(\sum_j |S_j| \cdot \log m\right)$ — ogni elemento contribuisce al più una volta agli aggiornamenti del guadagno, e ogni operazione sull'heap costa $O(\log m)$.

Garanzia di approssimazione: identica al greedy naive — $|\mathcal{C}| \leq H(\max_j |S_j|) \cdot \text{OPT}$. Il pattern Hash Map migliora solo la complessità, non la qualità della soluzione.

Test di validazione

La correttezza è verificata in `test/algoritmi/greedy_hash_pattern.py`

tramite `testGreedyHashMap()`. I cinque test producono le stesse cover del greedy naive sulle stesse istanze, confermando che l'ottimizzazione non altera la qualità della soluzione.

4.5 Randomized Rounding

Il **Randomized Rounding** è un algoritmo di approssimazione randomizzato per Set Cover basato sul rilassamento LP del problema ILP. È un algoritmo **Monte Carlo one-sided**: il tempo è deterministico, la soluzione può essere non valida con probabilità controllabile.

Fase 1 — Rilassamento LP

Si rilassa il vincolo di interezza $x_j \in \{0, 1\}$ a $x_j \in [0, 1]$:

$$\begin{aligned} \min \quad & \sum_{j=1}^m x_j \\ \text{s.t.} \quad & \sum_{j: e \in S_j} x_j \geq 1 \quad \forall e \in X, \quad x_j \in [0, 1] \quad \forall j \end{aligned}$$

La soluzione ottima $\hat{x}$ soddisfa $\text{OPT}_{\text{LP}} \leq \text{OPT}$ perché ogni soluzione intera è anche LP-ammissibile.

Fase 2 — Randomized Rounding

Si usa $\hat{x}$ come vettore di probabilità: si ripetono $t = \lceil c \ln n \rceil$ round indipendenti, includendo S_j con probabilità $\hat{x}_j$:

Algorithm 5: Randomized Rounding Set Cover (Monte Carlo) - pseudocodice

Input: universo X , famiglia $\mathcal{F}$, parametro $c \geq 1$, **seed**

Output: cover $\mathcal{C}$ (può essere non valida con prob. $\leq n^{1-c}$)

```

1  $\hat{x} \leftarrow \text{SOLVELP}(X, \mathcal{F});$ 
2  $t \leftarrow \lceil c \ln n \rceil;$ 
3  $\mathcal{C} \leftarrow \emptyset;$ 
4 for  $r = 1$  to  $t$  do
5   for  $j = 1$  to  $m$  do
6     if  $\text{RANDOM}(0, 1) < \hat{x}_j$  then
7        $\mathcal{C} \leftarrow \mathcal{C} \cup \{S_j\};$ 
8 return  $\mathcal{C};$ 
```

Garanzie teoriche

Qualità della soluzione: per linearità dell'aspettazione, ogni round include in media OPT_{LP} insiemi. Dopo t round:

$$\mathbb{E}[|\mathcal{C}|] \leq t \cdot \text{OPT}_{\text{LP}} \leq \lceil c \ln n \rceil \cdot \text{OPT}$$

Probabilità di fallimento: il vincolo LP garantisce che per ogni elemento $e \in X$:

$$\sum_{j: e \in S_j} \hat{x}_j \geq 1$$

Sfruttando questa garanzia si dimostra che la probabilità che e non venga coperto in un singolo round è al più $\frac{1}{e}$. Dopo $t = \lceil c \ln n \rceil$ round indipendenti e applicando la union bound su tutti gli n elementi:

$$\Pr[\text{fallimento}] \leq n \cdot n^{-c} = n^{1-c}$$

Con $c = 2$: $\Pr[\text{fallimento}] \leq \frac{1}{n}$ — garanzia w.h.p.

Bottleneck: gli esperimenti dimostrano che il LP occupa oltre il 90.0% del tempo totale. Il rounding e la verifica sono quindi trascurabili, come mostrato in Tabella 2 e Tabella 3

Tabella 2: Percentuale di tempo per fase — `workhorse/analisi.py`, `analisiBottleneckWH()`, $m=50$, $p=0.3$

Fase	Percentuale tempo
LP (solver)	94.0%
Rounding	0.3%
Verifica	5.7%

Tabella 3: Tempo medio per fase e n — `workhorse/analisi.py`, `analisiBottleneckWH()`, $m=50$, $p=0.3$, 5 seed

n	T_{LP} (s)	T_{round} (s)	T_{verifica} (s)
100	0.000561	0.000005	0.000017
200	0.001254	0.000005	0.000045
400	0.002891	0.000006	0.000114
800	0.005843	0.000008	0.000316
1600	0.010792	0.000007	0.000742
3200	0.024483	0.000009	0.001685
6400	0.046762	0.000010	0.003514
12800	0.099253	0.000011	0.007476

Test di validazione — Monte Carlo

La correttezza è verificata in `test/algoritmi/randomized.py` tramite `testRandomizedRounding()`. I test coprono quattro casi:

1. **TEST 1** — istanza semplice con $|X| = 5$, $m = 4$: cover trovata $\mathcal{C} = \{S_1, S_3\}$, valida.
2. **TEST 2** — istanza generata casualmente ($n = 10$, $m = 5$, $p = 0.4$, `seed=42`): stesso seed produce sempre stessa cover $\mathcal{C} = \{S_1, S_2, S_4\}$.
3. **TEST 3** — istanza worst-case ($k = 4$, $n = 30$): cover ottima trovata $\mathcal{C} = \{S_4, S_5\}$, $|\mathcal{C}| = 2 = \text{OPT}$.
4. **TEST 4** — tasso di successo su 100 esecuzioni con $c = 2$: 100/100 cover valide, confermando la garanzia w.h.p.

Variante Las Vegas

La variante **Las Vegas** risolve il LP una volta sola e ripete il rounding finché la cover è valida:

Algorithm 6: Randomized Rounding Set Cover (Las Vegas) - pseudocodice

Input: universo X , famiglia $\mathcal{F}$, seed
Output: cover $\mathcal{C}$ sempre valida

```

1  $\hat{x} \leftarrow \text{SOLVELP}(X, \mathcal{F});$ 
2 repeat
3    $\mathcal{C} \leftarrow \emptyset;$ 
4   for  $j = 1$  to  $m$  do
5     if  $\text{RANDOM}(0, 1) < \hat{x}_j$  then
6        $\mathcal{C} \leftarrow \mathcal{C} \cup \{S_j\};$ 
7 until  $\mathcal{C}$  è una cover valida;
8 return  $\mathcal{C};$ 
```

Costo temporale: Il tempo di esecuzione della variante Las Vegas è composto da due contributi:

$$T_{\text{LV}} = T_{\text{LP}} + T_{\text{rounding}}$$

T_{LP} — **deterministico**. La risoluzione del rilassamento lineare (LP) è eseguita **una volta sola** e produce la soluzione frazionaria $\hat{x}$. Il suo costo è deterministico e polinomiale rispetto alla dimensione del problema (n vincoli, m variabili): $T_{\text{LP}} = O(\text{Poly}(n, m))$.

T_{rounding} — **aleatorio**. La fase di rounding ripete il campionamento e la successiva verifica di ammissibilità finché la cover prodotta non è valida. Il numero di iterazioni è una **variabile aleatoria** con distribuzione geometrica di parametro p_s , dove p_s è la probabilità che un singolo round produca una cover valida:

$$\mathbb{E}[\text{iter}] = \frac{1}{p_s}$$

La probabilità p_s dipende strettamente dalla struttura della soluzione LP $\hat{x}$: quando i valori $\hat{x}_j$ sono piccoli, la probabilità che un singolo round copra contemporaneamente tutti gli n elementi decresce esponenzialmente. Di conseguenza, $\mathbb{E}[\text{iter}]$ cresce in modo non polinomiale rispetto a n .

Considerando che ogni tentativo richiede il rounding degli m insiemi e la verifica di copertura sugli n elementi, il costo totale atteso del rounding è:

$$\mathbb{E}[T_{\text{rounding}}] = \frac{1}{p_s} \cdot O(m + n)$$

La variante Las Vegas si rivela quindi un algoritmo **inefficiente** su istanze di grandi dimensioni poiché il suo tempo atteso non è polinomiale, a differenza della variante Monte Carlo che garantisce un tempo di esecuzione deterministico pari a $O(T_{\text{LP}} + t \cdot m)$.

Test di validazione — Las Vegas

La correttezza è verificata in

`test/algoritmi/randomized_las_vegas.py` tramite `testRandomizedRoundingLasVegas()`.

I test coprono quattro casi:

1. **TEST 1** — istanza semplice con $|X| = 5$, $m = 4$: cover valida trovata in 1 iterazione.
2. **TEST 2** — istanza generata casualmente ($n = 10$, $m = 5$, $p = 0.4$, **seed=99**): stesso seed produce sempre stessa cover $\mathcal{C} = \{S_1, S_2, S_4\}$ in 1 iterazione.
3. **TEST 3** — istanza worst-case ($k = 4$, $n = 30$): cover ottima $\mathcal{C} = \{S_4, S_5\}$ trovata in 1 iterazione.
4. **TEST 4** — tasso di successo su 100 esecuzioni: 100/100 cover valide, numero medio di iterazioni = 1.

Tabella 4: Confronto tra le due varianti del Randomized Rounding

Proprietà	Monte Carlo	Las Vegas
Ammissibilità	con prob. $1 - n^{1-c}$	sempre
Tempo	deterministico $O(T_{LP} + t \cdot m)$	aleatorio $O(T_{LP}) + \frac{1}{p_s} \cdot O(m + n)$
Terminazione	garantita	garantita con prob. 1
Iterazioni	$t = \lceil c \ln n \rceil$ fisse	aleatorio, media $\frac{1}{p_s}$
Efficienza	polinomiale	non polinomiale su istanze grandi

5 Strumenti e metodologia

Strumenti - Ambiente di esecuzione

Tabella 5: Configurazione hardware e software

Componente	Dettaglio
Hardware	MacBook Air M4
Memoria	16 GB
Sistema operativo	macOS Tahoe 26.5.1
IDE	VS Studio Code
Pilot	Python 3.14, ambiente virtuale (.venv)
Solver LP (pilot)	PuLP 3.3.2 con backend CBC
Solver ILP (pilot)	Gurobi
Librerie	pandas, matplotlib
Workhorse	C++17, clang++ con flag -O2
Solver LP (workhorse)	GLPK 5.0 (via Homebrew)
Misurazione tempo	<code>std::clock()</code> (CPU time)

Algorithm Engineering

Gli esperimenti seguono la metodologia dell'Algorithm Engineering, divisi in due fasi complementari:

- **Pilot** (Python) — studio esplorativo su istanze piccole e medie. Obiettivi: verificare la correttezza, identificare i bottleneck, scoprire le ipotesi da validare, calibrare i parametri per il workhorse. I risultati del pilot non sono conclusioni finali.

Gli esperimenti pilot che su larga scala non apportano ulteriori elementi allo studio del Problema non sono stati riprodotti nel Workhorse.

- **Workhorse** (C++) — studio sistematico su istanze grandi, con misurazioni accurate di CPU time. Obiettivi: validare le ipotesi del pilot su larga scala, stimare i modelli di costo, produrre conclusioni solide e replicabili.

Parametri degli esperimenti

Tabella 6: Parametri pilot e workhorse

Parametro	Pilot	Workhorse
n (universo)	$5 \div 1280$	$100 \div 51200$
m (insiemi)	$5 \div 100$	20, 50, 100
p (densità)	$0.1 \div 0.5$	0.1, 0.3, 0.5
seed	$200 \div 500$	10
c (RR)	1, 2, 3, 4	2

Metriche misurate

Le metriche sono scelte per catturare sia la **qualità** della soluzione che le **prestazioni** degli algoritmi.

- Il **CPU time** è preferito al wall clock time perché misura il tempo effettivamente consumato dal processo sul processore, escludendo attese dovute al sistema operativo, I/O o altri processi in esecuzione concorrente. Questo garantisce misurazioni più accurate e replicabili, indipendenti dal carico del sistema. Nel `workhorse` è misurato con `std::clock()`.
- **Rapporto di approssimazione** $\rho = |\mathcal{C}|/\text{OPT}$ — qualità della soluzione rispetto all'ottimo.
- **Tasso di successo** — percentuale di esecuzioni che producono una cover valida (rilevante per il Randomized Rounding Monte Carlo).
- **Dimensione della cover** $|\mathcal{C}|$ — numero di insiemi selezionati.

Riproducibilità

Tutti gli esperimenti sono progettati per essere completamente riproducibili. La riproducibilità è garantita da tre elementi:

- **Seed fisso** — ogni esperimento utilizza un insieme predefinito di seed per il generatore di numeri casuali. Lo stesso seed produce sempre la stessa istanza e la stessa sequenza di scelte casuali dell'algoritmo.
- **Generatore deterministico** — il generatore Bernoulli utilizza `random.Random(seed)` in Python e `std::mt19937(seed)` in C++, entrambi deterministici dato il seed.
- **Parametri espliciti** — tutti i parametri degli esperimenti (n , m , p , seed , c) sono documentati nelle caption delle tabelle e negli script di esecuzione, permettendo la replica esatta dei risultati.

6 Pilot

La fase pilot è implementata in Python nella cartella `test/`. Gli esperimenti sono esplorativi, su istanze piccole e medie, con l'obiettivo di verificare la correttezza, identificare i bottleneck e calibrare i parametri per il workhorse.

6.1 Implementazione degli algoritmi

Gli algoritmi sono implementati nella cartella `test/algoritmi/`:

Tabella 7: File di implementazione degli algoritmi – pilot Python

File	Contenuto	Dipendenze
<code>utils.py</code>	Generatore Bernoulli, validatore	<code>random</code> , <code>itertools</code>
<code>brute_force.py</code>	Brute Force	<code>utils.py</code>
<code>ilp.py</code>	ILP	Gurobi, <code>utils.py</code>
<code>greedy.py</code>	Greedy naive	<code>utils.py</code>
<code>greedy_hash_pattern.py</code>	Greedy Hash Map	<code>utils.py</code>
<code>lp_solver.py</code>	LP relaxation	PuLP 3.3.2
<code>randomized.py</code>	Randomized Rounding (MC)	<code>lp_solver.py</code> , <code>utils.py</code>
<code>randomized_las_vegas.py</code>	Randomized Rounding (LV)	<code>lp_solver.py</code> , <code>utils.py</code>

6.2 Test di validazione

La correttezza di tutti gli algoritmi è verificata tramite il file `test/regression_test.py`, che esegue in sequenza tutti i test di validazione. La Tabella 8 riepiloga i risultati.

Tabella 8: Riepilogo test di validazione – `regression_test.py`

Algoritmo	Funzione di test	N. test	Esito
Generatore Bernoulli	<code>testInstanceGenerators()</code>	4	✓
Brute Force	<code>testBruteForce()</code>	4	✓
ILP	<code>testILP()</code>	3	✓
Greedy naive	<code>testGreedy()</code>	5	✓
Greedy Hash Map	<code>testGreedyHashMap()</code>	5	✓
Randomized RR (MC)	<code>testRandomizedRounding()</code>	4	✓
Randomized RR (LV)	<code>testRandomizedRoundingLasVegas()</code>	4	✓

Tutti i test implementati passano senza errori. In particolare:

Generatore Bernoulli – testInstanceGenerators()

Tabella 9: Test di validazione – `utils.py`, `testInstanceGenerators()`

Test	Input	Output	Esito	Conclusione
TEST 1	$n = 10, m = 4, p = 0.0$	$ S_j \geq 1$ per ogni j	✓	Copertura garantita anche con $p = 0$
TEST 2	$n = 10, m = 4, p \in \{0.1, \dots, 0.9\}$, seed=123	media $ S_j $ cresce con p	✓	$\mathbb{E}[S_j] = n \cdot p$ verificato
TEST 3	$n = 10, m = 4, p = 0.5$, seed variabile	istanze diverse per seed diversi	✓	Generatore non degenerare
TEST 4	$n = 10, m = 4, p = 0.5$, seed=42 (x2)	istanze identiche	✓	Riproducibilità garantita

Brute Force – testBruteForce()

Tabella 10: Test di validazione – `brute_force.py`, `testBruteForce()`

Test	Input	Output	Esito	Conclusione
TEST 1	$ X = 5, m = 4$	$ \mathcal{C}^* = 2$	✓	Ottimo trovato
TEST 2	$ X = 3, m = 3$	$ \mathcal{C}^* = 1$	✓	Ottimo trovato
TEST 3	$n = 10, m = 5, p = 0.4$, seed=42	$ \mathcal{C}^* = 3$, valida	✓	Cover valida e ottima
TEST 4	worst-case $k = 4$, $n = 30$	$ \mathcal{C}^* = 2$, $\mathcal{C}^* = \{S_4, S_5\}$	✓	Ottimo trovato

ILP – testILP()

Tabella 11: Test di validazione – `ilp.py`, `testILP()`

Test	Input	Output	Esito	Conclusione
TEST 1	$ X = 5, m = 4$	$ \mathcal{C}^* = 2$, $\mathcal{C}^* = \{S_1, S_3\}$	✓	Ottimo trovato
TEST 2	$n = 10, m = 5, p = 0.4$, seed=42	$ \mathcal{C}^* = 3$, valida	✓	Cover valida e ottima
TEST 3	worst-case $k = 4$, $n = 30$	$ \mathcal{C}^* = 2$, $\mathcal{C}^* = \{S_4, S_5\}$	✓	Ottimo trovato

Greedy naive – testGreedy()

Tabella 12: Test di validazione – greedy.py, testGreedy()

Test	Input	Output	Esito	Conclusione
TEST 1	$ X = 5, m = 4$	$ \mathcal{C} = 2 = \text{OPT}$	✓	Cover ottima
TEST 2	$ X = 3, m = 3$	$ \mathcal{C} = 1 = \text{OPT}$	✓	Cover ottima
TEST 3	$n = 10, m = 5, p = 0.4, \text{seed}=42$	cover valida	✓	Cover valida
TEST 4	$n = 20, m = 8, p = 0.5, \text{seed}=42$	$ \mathcal{C} = 3, \rho \leq H(14)$	✓	Bound rispettato
TEST 5	worst-case $k = 4, n = 30$	$ \mathcal{C} = 4, \rho = 2.0$	✓	Comportamento atteso

Greedy Hash Map – testGreedyHashMap()

Tabella 13: Test di validazione – greedy_hash_pattern.py, testGreedyHashMap()

Test	Input	Output	Esito	Conclusione
TEST 1	$ X = 5, m = 4$	$ \mathcal{C} = 2$, identica al greedy	✓	Stessa qualità
TEST 2	$ X = 3, m = 3$	$ \mathcal{C} = 1$, identica al greedy	✓	Stessa qualità
TEST 3	$n = 10, m = 5, p = 0.4, \text{seed}=42$	cover valida	✓	Cover valida
TEST 4	$n = 20, m = 8, p = 0.5, \text{seed}=42$	$ \mathcal{C} = 3, \rho \leq H(14)$	✓	Bound rispettato
TEST 5	worst-case $k = 4, n = 30$	$ \mathcal{C} = 4, \rho = 2.0$	✓	Comportamento identico al greedy

Randomized Rounding Monte Carlo – testRandomizedRounding()

Tabella 14: Test di validazione – randomized.py, testRandomizedRounding()

Test	Input	Output	Esito	Conclusione
TEST 1	$ X = 5, m = 4, \text{seed}=42$	$\mathcal{C} = \{S_1, S_3\}$, valida	✓	Cover valida
TEST 2	$n = 10, m = 5, p = 0.4, \text{seed}=99$ (x2)	stessa cover entrambe le volte	✓	Riproducibilità garantita
TEST 3	worst-case $k = 4, n = 30$	$\mathcal{C} = \{S_4, S_5\}, \mathcal{C} = 2$	✓	Cover ottima sul worst-case
TEST 4	$n = 15, m = 6, p = 0.4, 100 \text{ seed}, c = 2$	100/100 cover valide	✓	Tasso successo 100%

Randomized Rounding Las Vegas – testRandomizedRoundingLasVegas()

Tabella 15: Test di validazione – randomized_las_vegas.py, testRandomizedRoundingLasVegas()

Test	Input	Output	Esito	Conclusione
TEST 1	$ X = 5, m = 4, \text{seed}=42$	$\mathcal{C} = \{S_1, S_3\}, 1 \text{ iter.}$	✓	Cover valida in 1 iterazione
TEST 2	$n = 10, m = 5, p = 0.4, \text{seed}=99 \text{ (x2)}$	stessa cover, stesse iter.	✓	Riproducibilità garantita
TEST 3	worst-case $k = 4, n = 30$	$\mathcal{C} = \{S_4, S_5\}, 1 \text{ iter.}$	✓	Cover ottima in 1 iterazione
TEST 4	$n = 15, m = 6, p = 0.4, 100 \text{ seed}$	100/100 cover valide, iter. media = 1	✓	Correttezza garantita

6.3 Esperimenti

Gli esperimenti pilot sono condotti su istanze piccole e medie con l'obiettivo di esplorare il comportamento degli algoritmi, identificare i bottleneck e formulare ipotesi. Alcuni esperimenti sono stati successivamente riprodotti e validati su larga scala nella fase workhorse in C++ con istanze di dimensione maggiore.

Ogni esperimento è implementato in uno script Python dedicato nella cartella `test/esperimenti/`, che esegue le misurazioni e salva i risultati in un file CSV nella cartella `test/risultati/`. Il file `analisi.py` elabora i CSV producendo tabelle di sintesi e statistiche descrittive. Il file `plot.py` genera i grafici a partire dai dati analizzati, salvandoli in `test/risultati/plot/`.

Il flusso è quindi:

`exp_*.py → *.csv → analisi.py → plot.py`

6.3.1 Esperimento 1 — Qualità della soluzione

Domanda sperimentale: come varia il rapporto di approssimazione $\rho = |\mathcal{C}|/\text{OPT}$ al variare dell'algoritmo, della densità p e della dimensione n ?

Parametri: $n \in \{5, 10, 15, 20\}$, $m = 10$, $p \in \{0.2, 0.3, 0.5\}$, 10 seed, $c = 2$.

Tabella 16: Rapporto di approssimazione medio per algoritmo e p – exp_qs.py, TestQual()

Algoritmo	$p = 0.2$	$p = 0.3$	$p = 0.5$
Brute Force	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
ILP	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
Greedy	1.010 ± 0.045	1.054 ± 0.123	1.013 ± 0.079
Randomized	1.045 ± 0.174	1.203 ± 0.379	1.472 ± 0.602

Tabella 17: Rapporto di approssimazione medio per algoritmo e n – `exp_qs.py`, `TestQual()`

Algoritmo	$n = 5$	$n = 10$	$n = 15$	$n = 20$
Brute Force	1.000	1.000	1.000	1.000
ILP	1.000	1.000	1.000	1.000
Greedy	1.033	1.028	1.016	1.024
Randomized	1.136	1.257	1.254	1.311

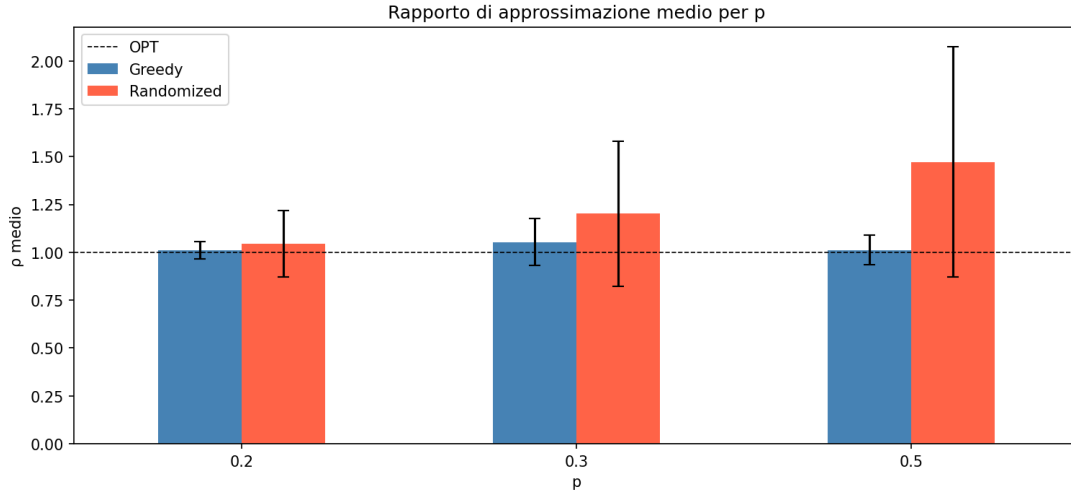


Figura 2: Rapporto di approssimazione medio per algoritmo e p – `exp_qs.py`, `TestQual()`

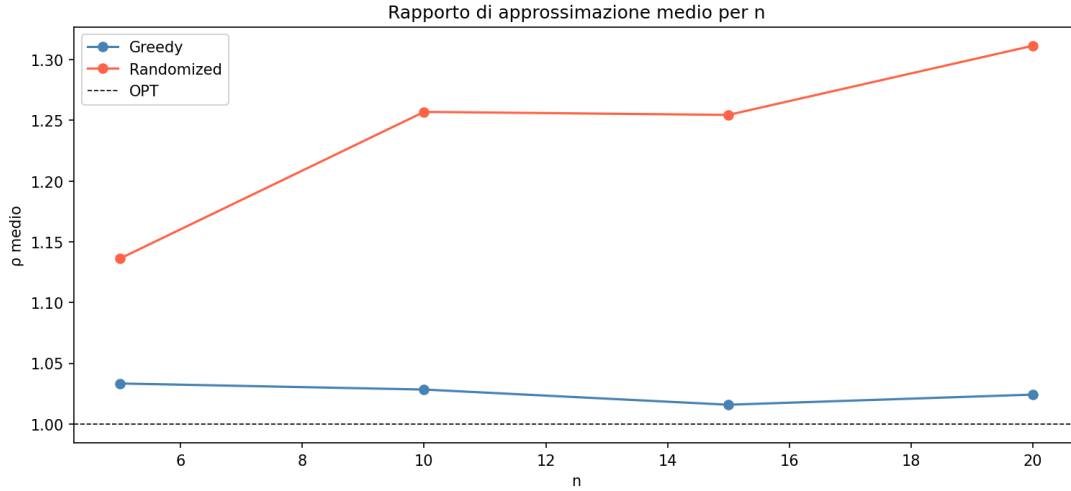


Figura 3: Rapporto di approssimazione medio per algoritmo e n – `exp_qs.py`, `TestQual()`

Conclusioni: Brute Force e ILP trovano sempre l'ottimo ($\rho = 1$), confermando la loro correttezza come riferimento. Il Greedy è quasi ottimo in pratica su istanze casuali ($\rho \approx 1.01 \div 1.05$) — molto al di sotto del bound teorico $H(\max |S|) \leq \ln n + 1$. Il Randomized Rounding accumula ridondanza su istanze casuali — ρ cresce con p e con n , raggiungendo 1.47 per $p = 0.5$. Questo è coerente con la teoria: la garanzia

$\mathbb{E}[|C|] \leq \lceil c \ln n \rceil \cdot \text{OPT}$ è in aspettazione — il singolo run può produrre cover più grandi dell’ottimo. Su istanze casuali il Greedy deterministico domina il Randomized in termini di qualità. I casi in cui il Randomized supera il Greedy sono 65 su 240 (27%) — il vantaggio strutturale del Randomized emerge su input avversariali, non su istanze casuali (Exp. 3).

6.3.2 Esperimento 2 — Runtime: Greedy vs Randomized Rounding

Domanda sperimentale: come scala il tempo di esecuzione di Greedy e Randomized Rounding al variare di n ?

Parametri: $n \in \{50, 100, 200, 400, 800\}$, $m = 50$, $p = 0.3$, 10 seed, $c = 2$.

Nota: questo esperimento è stato riprodotto su larga scala nella fase workhorse con istanze fino a $n = 51200$ e misurazioni accurate di CPU time in C++.

Tabella 18: Tempo medio di esecuzione (s) per $n - \text{exp_rt.py}$, $\text{TestRunTime}()$, $m=50$, $p=0.3$

n	Greedy (s)	Randomized (s)
50	0.000117	0.013008
100	0.000270	0.015669
200	0.000574	0.022232
400	0.001374	0.039360
800	0.003091	0.071980

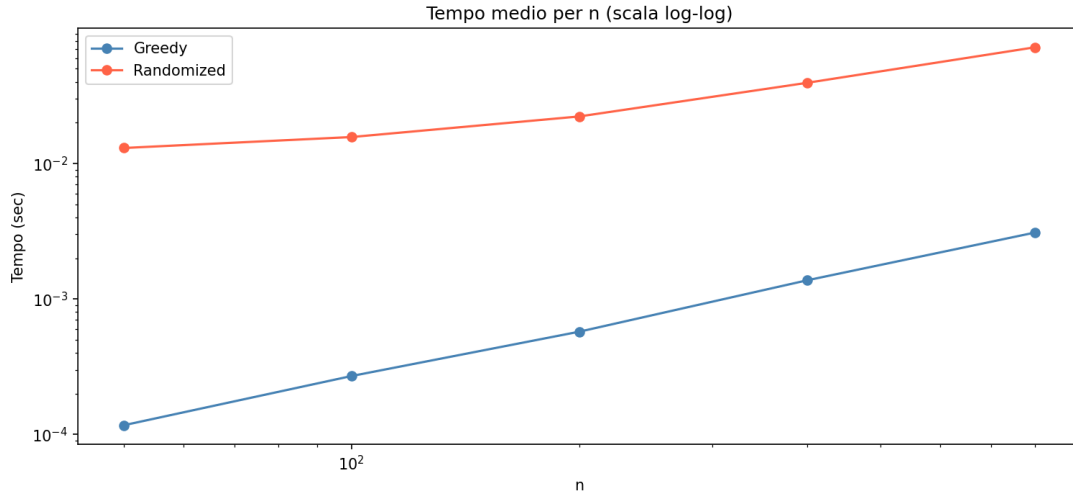


Figura 4: Tempo medio per n in scala log-log – exp_rt.py , $\text{TestRunTime}()$, $m=50$, $p=0.3$

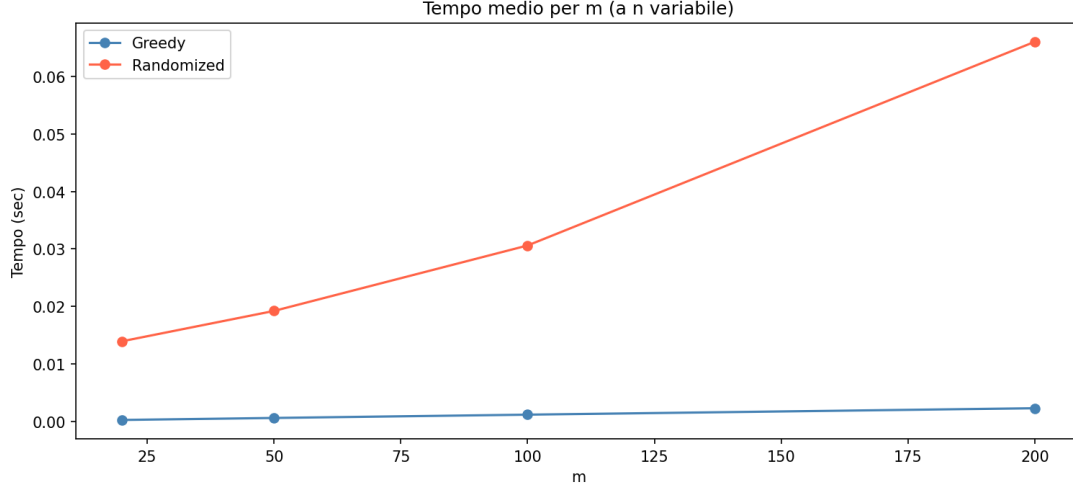


Figura 5: Tempo medio per m – `exp_rt.py`, `TestRunTime()`, $p=0.3$

Conclusioni: il Greedy è 20–100x più veloce del Randomized Rounding su istanze casuali. Il bottleneck del Randomized è il solver LP (PuLP/CBC), che occupa il 99.9% del tempo totale (Exp. 9) — coerente con il modello di costo teorico $T_{MC} = O(T_{LP} + t \cdot m)$ dove T_{LP} domina. Il rounding $O(t \cdot m) = O(\ln n \cdot m)$ e la verifica $O(n)$ sono trascurabili. Nella fase *workhorse* con GLPK nativo questo overhead è drasticamente ridotto — il Randomized diventa comparabile al Greedy per n grandi (Exp. W1).

6.3.3 Esperimento 3 — Greedy Worst Case

Domanda sperimentale: come si comportano gli algoritmi sull’istanza avversariale costruita appositamente per ingannare il Greedy?

Parametri: $k \in \{3, 4, 5, 6, 7, 8\}$, istanza worst-case generata da `generate_worst_case(k)`.

L’istanza worst-case per k ha universo di dimensione $n = 2^{k+1} - 2$ e $2k$ insiemi: k insiemi “piccoli” $S_0, \dots, S_{k-1}$ di dimensioni $2, 4, 8, \dots, 2^k$, e due insiemi ottimi S_k, S_{k+1} che coprono tutto l’universo con $|\mathcal{C}^*| = 2$.

Tabella 19: Rapporto di approssimazione sul worst-case per k – `exp_wc.py`, `TestWorstCase()`

k	n	Brute Force	ILP	Greedy	Randomized
3	14	1.0	1.0	1.5	1.0
4	30	1.0	1.0	2.0	1.0
5	62	1.0	1.0	2.5	1.0
6	126	1.0	1.0	3.0	1.0
7	254	1.0	1.0	3.5	1.0
8	510	1.0	1.0	4.0	1.0

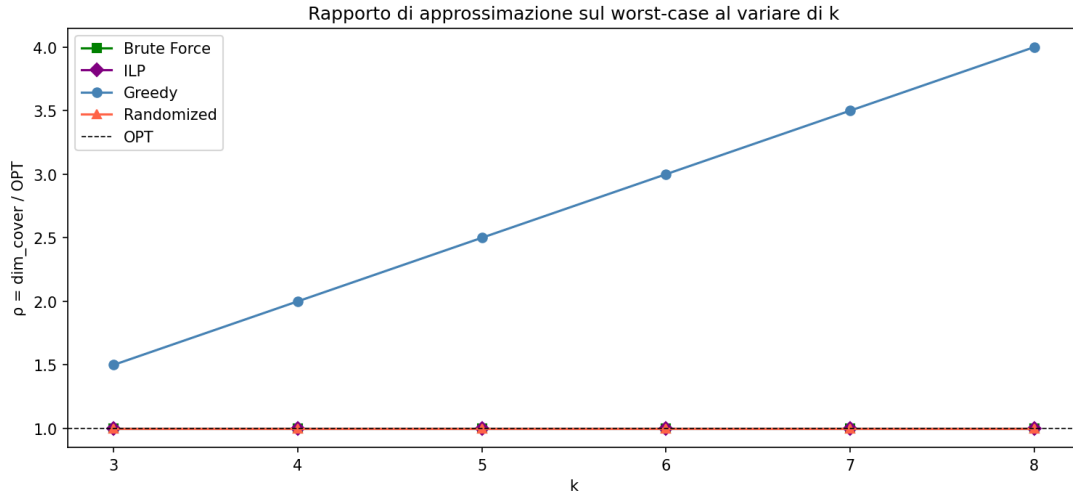


Figura 6: Rapporto di approssimazione sul worst-case – `exp_wc.py`, `TestWorstCase()`

Conclusioni: sull’istanza avversariale il Greedy raggiunge $\rho = k/2$ — crescita lineare in k , confermando che la garanzia teorica $\rho \leq H(\max |S|) \leq \ln n + 1$ è tight su questa famiglia di istanze. Il Greedy fallisce perché la scelta *locally optimal* (massimo guadagno immediato) non è *globally optimal* — seleziona gli insiemi piccoli $S_0, \dots, S_{k-1}$ invece dei due insiemi ottimi S_k, S_{k+1} .

Il Randomized Rounding trova sempre l’ottimo ($\rho = 1$) — la soluzione LP assegna peso $\frac{1}{2}$ a S_k e S_{k+1} , e il rounding li seleziona con alta probabilità. Questo è il vantaggio strutturale della randomizzazione: la casualità interna rende l’algoritmo robusto agli input avversariali costruiti per ingannare le euristiche deterministiche.

6.3.4 Esperimento 4 — Variabilità della soluzione

Domanda sperimentale: quanto varia la dimensione della cover $|\mathcal{C}|$ tra esecuzioni diverse sullo stesso input, al variare del seed?

Parametri: $n = 50$, $m = 20$, $p = 0.3$, 200 seed, $c = 2$. Istanza fissa, seed variabile — si isola la variabilità dell’algoritmo da quella dell’istanza.

Tabella 20: Statistiche di $|\mathcal{C}|$ per algoritmo – `exp3_var.py`, `varSeedRand()`, $n=50$, $m=20$, $p=0.3$, 200 seed

Algoritmo	media	std	min	max	valori unici
Greedy	9.00	0.00	9	9	1
Randomized	25.27	2.39	19	31	13

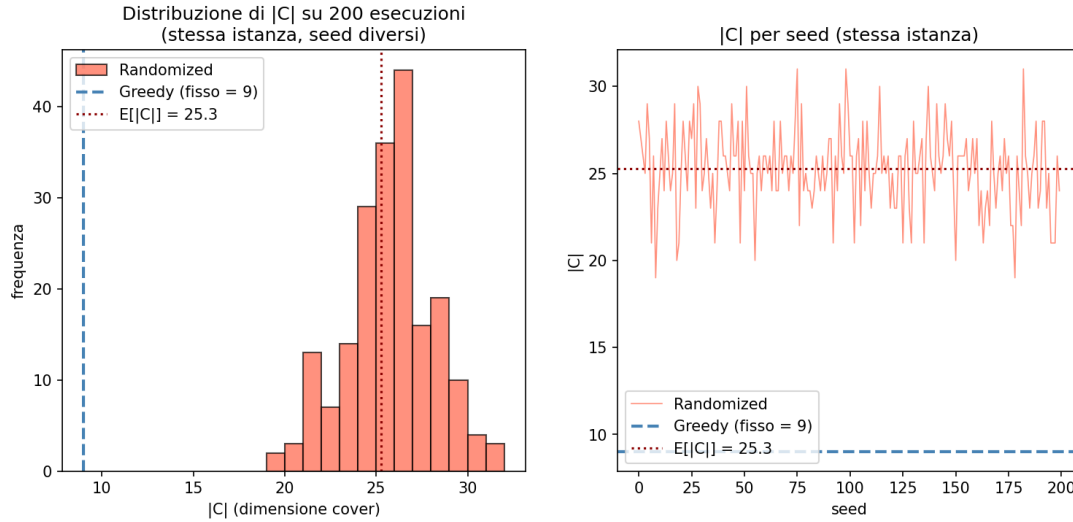


Figura 7: Distribuzione di $|C|$ al variare del seed – `exp3_var.py`, `varSeedRand()`, $n=50$, $m=20$, $p=0.3$

Conclusioni: il Greedy è completamente deterministico ($\text{std} = 0$, unico valore possibile $|C| = 9$) — produce sempre la stessa cover sullo stesso input. Il Randomized Rounding è stocastico: la distribuzione di $|C|$ è approssimativamente gaussiana, con media 25.27 e deviazione standard 2.39. Il range $[19, 31]$ mostra che la cover può variare significativamente tra esecuzioni diverse, pur rimanendo concentrata intorno alla media — confermando la garanzia teorica $\mathbb{E}[|C|] = O(\ln n) \cdot \text{OPT}$.

6.3.5 Esperimento 5 — Effetto del parametro c nel Rand. Round.

Domanda sperimentale: come influisce il parametro c sul numero di round $t = \lceil c \ln n \rceil$, sul tasso di successo e sulla dimensione media della cover nel Randomized Rounding (Monte Carlo)?

Parametri: $n = 50$, $m = 20$, $p = 0.3$, 200 seed, $c \in \{1, 2, 3, 4\}$.

Tabella 21: Effetto del parametro c – `exp5_param_c.py`, `nRoundRand()`, $n=50$, $m=20$, $p=0.3$, 200 seed

c	$t = \lceil c \ln n \rceil$	tasso successo	media $ C $	std $ C $
1	4	88.5%	9.82	1.29
2	8	99.5%	11.41	1.09
3	12	100.0%	12.03	0.97
4	16	100.0%	12.50	0.89

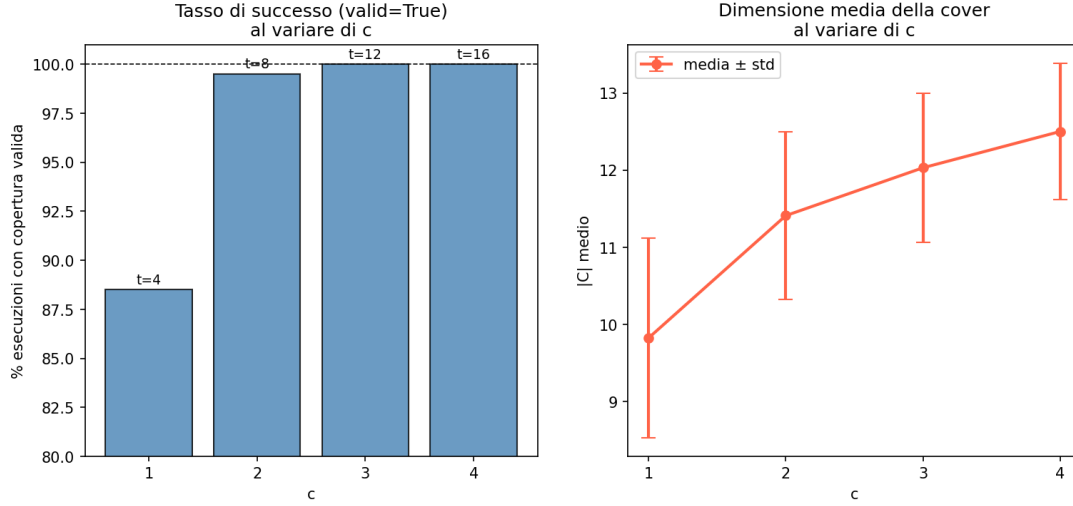


Figura 8: Effetto del parametro c su tasso di successo e $|\mathcal{C}|$ — `exp5_param_c.py`, `nRoundRand()`, $n=50$, $m=20$, $p=0.3$

Conclusioni: i dati confermano il trade-off teorico fondamentale del Randomized Rounding: aumentare c è **costoso in aspettazione** — $\mathbb{E}[|\mathcal{C}|] \leq \lceil c \ln n \rceil \cdot \text{OPT}$ cresce linearmente con c — ma **vantaggioso in probabilità** — $\Pr[\text{fallimento}] \leq n^{1-c}$ decresce esponenzialmente.

$c = 2$ è il punto di equilibrio ottimale: garantisce $\Pr[\text{fallimento}] \leq \frac{1}{n}$ (garanzia w.h.p.) con overhead logaritmico sulla dimensione della cover. Aumentare c oltre 2 porta benefici marginali — 100% di successo già da $c = 3$ — a scapito di cover progressivamente più grandi. Il parametro c è quindi l'unico strumento per controllare il trade-off tra affidabilità e qualità della soluzione.

6.3.6 Esperimento 6 — Tasso di fallimento empirico vs bound teorico

Domanda sperimentale: il bound teorico $\Pr[\text{fallimento}] \leq n^{1-c}$ è rispettato empiricamente? Quanto è conservativo?

Parametri: $n \in \{10, 20, 40, 80, 160, 320\}$, $m = 50$, $p = 0.3$, $c \in \{1, 2, 3\}$, 500 seed.

Tabella 22: Tasso di fallimento empirico vs bound teorico n^{1-c} – `exp6_fail_rate.py`, `randFailRate()`, $m=50$, $p=0.3$, 500 seed

n	c	t	fallimento emp.	bound teo. n^{1-c}	ratio
10	1	3	0.1220	1.0000	0.122
10	2	5	0.0200	0.1000	0.200
10	3	7	0.0020	0.0100	0.200
20	1	3	0.1240	1.0000	0.124
20	2	6	0.0040	0.0500	0.080
20	3	9	0.0000	0.0025	0.000
40	1	4	0.0620	1.0000	0.062
40	2	8	0.0000	0.0250	0.000
80	1	5	0.0560	1.0000	0.056
80	2	9	0.0000	0.0125	0.000
160	1	6	0.0060	1.0000	0.006
320	1	6	0.0120	1.0000	0.012

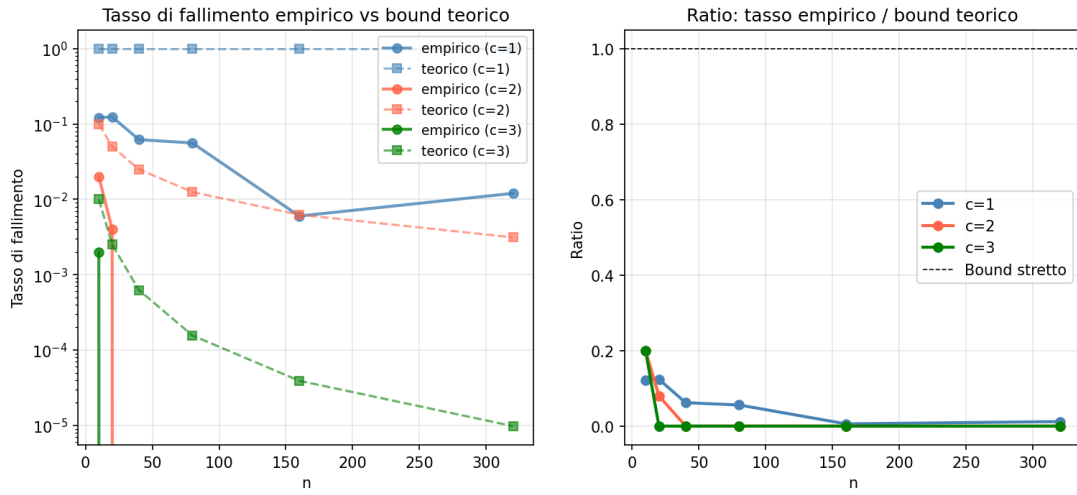


Figura 9: Tasso di fallimento empirico vs bound teorico (scala log) – `exp6_fail_rate.py`, `randFailRate()`, $m=50$, $p=0.3$

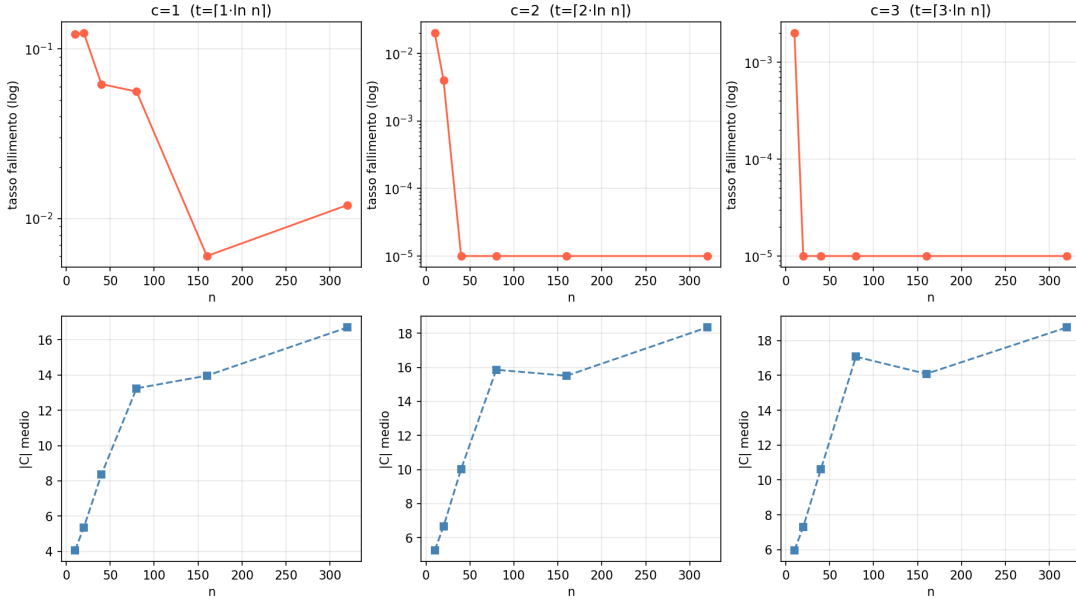


Figura 10: Trade-off tasso di fallimento vs $|\mathcal{C}|$ medio per $c = \text{exp6_fail_rate.py}$, $\text{randFailRate}()$, $m=50$, $p=0.3$

Conclusioni: il bound n^{1-c} è sempre rispettato empiricamente (ratio ≤ 1 in tutti i casi), ma è molto conservativo — il tasso reale è sistematicamente molto inferiore alla garanzia teorica. Questo è una conseguenza diretta del metodo di dimostrazione: la **union bound** somma le probabilità di fallimento per ogni elemento $e \in X$ assumendo il caso peggiore, ma nella pratica gli elementi tendono a essere coperti insieme, rendendo il bound non tight. Con $c = 2$ il fallimento si azzerà già per $n = 40$, confermando che la garanzia w.h.p. $\Pr[\text{fallimento}] \leq \frac{1}{n}$ è molto più forte di quanto il bound suggerisca in pratica. Il trade-off tra affidabilità e dimensione della cover è asimmetrico e favorevole: passare da $c = 1$ a $c = 2$ elimina quasi tutti i fallimenti a costo di pochi insiemi aggiuntivi — coerente con la crescita lineare di $\mathbb{E}[|\mathcal{C}|] \leq \lceil c \ln n \rceil \cdot \text{OPT}$.

6.3.7 Esperimento 7 — Distribuzione di $|\mathcal{C}|$ al variare di n

Domanda sperimentale: come si distribuisce la dimensione della cover $|\mathcal{C}|$ prodotta dal Randomized Rounding al variare di n ? La distribuzione è concentrata intorno alla media?

Parametri: $n \in \{20, 40, 80, 160, 320\}$, $m = 50$, $p = 0.3$, 500 seed, $c = 2$. Istanza fissa per ogni n .

Tabella 23: Statistiche di $|\mathcal{C}|$ per n — exp7_distrib.py , $\text{distribCover}()$, $m=50$, $p=0.3$, 500 seed, $c=2$

n	media	std	min	max	valid%
20	7.97	1.56	3	13	98.0%
40	13.18	1.85	9	19	100.0%
80	17.81	2.39	11	25	100.0%
160	24.06	2.62	17	34	100.0%
320	28.59	2.63	22	36	100.0%

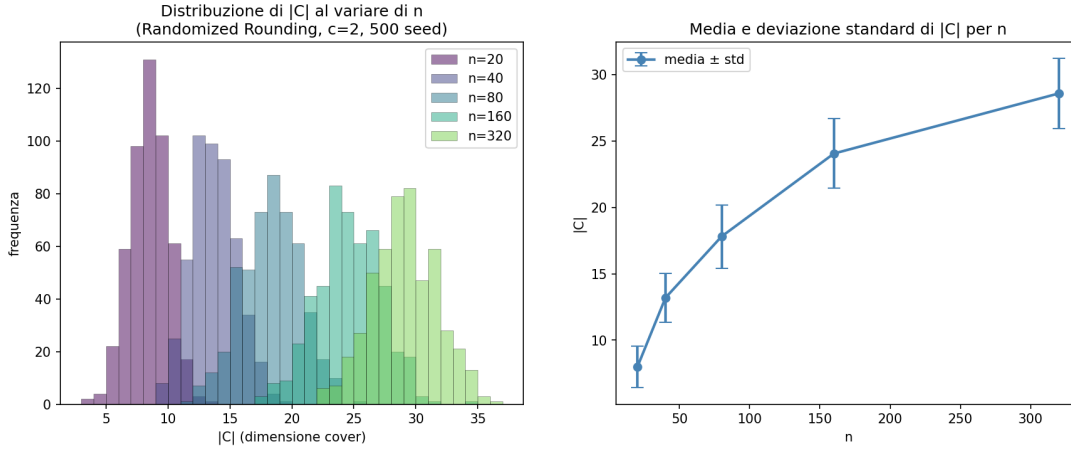


Figura 11: Distribuzione di $|C|$ al variare di n – `exp7_distrib.py`, `distribCover()`, $m=50$, $p=0.3$, 500 seed

Conclusioni: la distribuzione di $|C|$ è approssimativamente gaussiana e concentrata intorno alla media per ogni n . La media cresce in modo **sublogaritmico** con n — da 7.97 a 28.59 mentre n cresce da 20 a 320 (fattore 16) — coerente con la garanzia teorica $\mathbb{E}[|C|] = O(\ln n) \cdot \text{OPT}$. La deviazione standard cresce leggermente ma proporzionalmente decresce rispetto alla media: la randomizzazione diventa più stabile su istanze grandi. Il tasso di successo è 100% già da $n = 40$ con $c = 2$.

6.3.8 Esperimento 8 — Speedup del pattern Hash Map

Domanda sperimentale: il pattern Reduce-to-Hash-Tables migliora il runtime del Greedy? Lo speedup dipende dalla densità p e dalla dimensione n ?

Parametri: $n \in \{100, 200, 400, 800, 1600, 3200, 6400, 12800\}$, $m \in \{50, 100, 200\}$, $p \in \{0.1, 0.3, 0.5\}$, 10 seed.

Nota: questo esperimento è stato riprodotto su larga scala nella fase workhorse (Sezione 7), con istanze fino a $n = 51200$ e misurazioni accurate di CPU time in C++.

Tabella 24: Tempo medio (s) per n – `exp_hashmap.py`, `hashMapGreedy()`, $m=50$, $p=0.3$, 10 seed

n	Greedy (s)	Greedy Hash Map (s)	Speedup
100	0.000394	0.000319	1.23x
200	0.000890	0.000570	1.56x
400	0.002124	0.001046	2.03x
800	0.004976	0.002038	2.44x
1600	0.011235	0.003940	2.85x
3200	0.025782	0.010315	2.50x
6400	0.058350	0.021931	2.66x
12800	0.127826	0.048191	2.65x

Tabella 25: Tempo medio (s) e speedup per p – `exp_hashmap.py`, `hashMapGreedy()`, 10 seed

p	Greedy (s)	Greedy Hash Map (s)	Speedup
0.1	0.031899	0.004814	6.63x
0.3	0.027834	0.010695	2.60x
0.5	0.027108	0.017623	1.54x

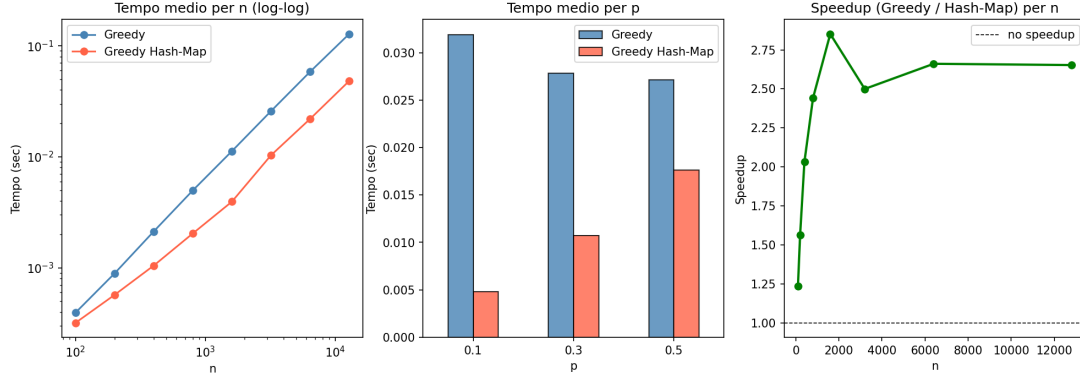


Figura 12: Speedup Greedy Hash Map vs Greedy naive per n e p – `exp_hashmap.py`, `hashMapGreedy()`

Conclusioni: il pattern Reduce-to-Hash-Tables porta un vantaggio reale e misurabile. Lo speedup cresce con n — da 1.23x a 2.85x — confermando il vantaggio asintotico di $O(\sum |S_j| \cdot \log m)$ rispetto a $O(n \cdot m)$. Il dato più significativo è la dipendenza da p : su istanze sparse ($p = 0.1$) lo speedup raggiunge 6.63x, mentre su istanze dense ($p = 0.5$) scende a 1.54x. Questo è teoricamente atteso: con p piccola $\sum |S_j|$ è piccolo e la mappa inversa aggiorna pochi insiemi per ogni elemento coperto.

6.3.9 Esperimento 9 — Las Vegas vs Monte Carlo

Domanda sperimentale: quante iterazioni richiede la variante Las Vegas rispetto ai round fissi $t = \lceil 2 \ln n \rceil$ del Monte Carlo? Qual è il trade-off in termini di dimensione della cover?

Parametri: $n \in \{50, 100, 200, 400, 800\}$, $m = 50$, $p = 0.3$, 500 seed, $c = 2$ per il Monte Carlo.

Tabella 26: Iterazioni Las Vegas vs round fissi Monte Carlo per n – `exp_rand_las_vegas.py`, `analisiLVvsMC()`, $m=50$, $p=0.3$, 500 seed

n	LV media	LV std	LV min	LV max	MC round fissi
50	57.9	61.1	1	386	8
100	315.6	337.2	1	2213	10
200	1553.4	1704.9	3	15324	11
400	7381.1	7643.9	10	46972	12
800	36472.3	36236.3	275	245555	14

Tabella 27: Tasso di successo Monte Carlo e dimensione media cover per n — `exp_rand_las_vegas.py`, `analisiLVvsMC()`, $m=50$, $p=0.3$, 500 seed

n	MC successo	LV $ \mathcal{C} $ medio	MC $ \mathcal{C} $ medio
50	98.6%	7.10	14.67
100	99.8%	9.06	20.49
200	100%	11.00	25.15
400	100%	12.97	29.18
800	100%	14.79	33.75

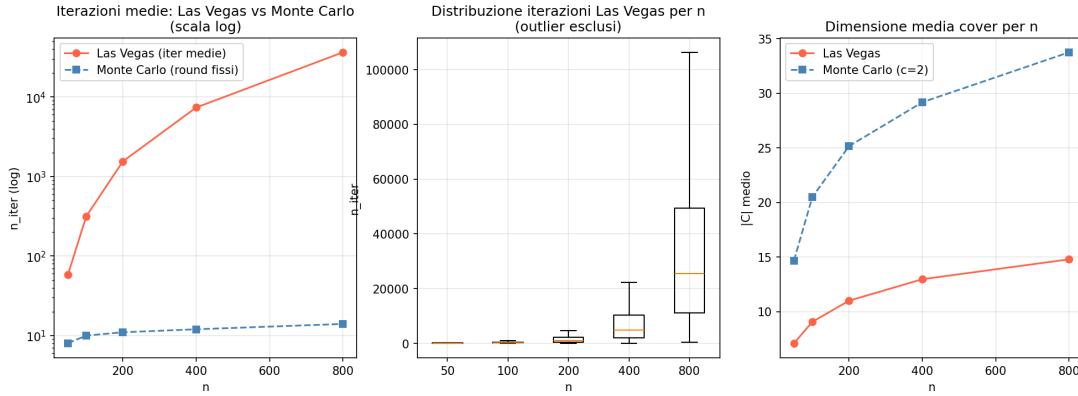


Figura 13: Las Vegas vs Monte Carlo: iterazioni, distribuzione e qualità — `exp_rand_las_vegas.py`, `analisiLVvsMC()`, $m=50$, $p=0.3$

Conclusioni: il confronto rivela un trade-off netto tra le due varianti.

Il Monte Carlo con $c = 2$ usa $t = \lceil 2 \ln n \rceil$ round fissi — tempo deterministico $O(T_{LP} + t \cdot m)$ — con tasso di successo $\geq 98.6\%$, confermando la garanzia w.h.p. $\Pr[\text{successo}] \geq 1 - \frac{1}{n}$.

La Las Vegas garantisce correttezza assoluta ma il numero di iterazioni è una variabile aleatoria geometricamente distribuita con media $\frac{1}{p_s}$. I dati mostrano che p_s è molto piccola su queste istanze: per $n = 800$ servono in media 36.472 iterazioni — tre ordini di grandezza in più rispetto ai 14 round fissi del Monte Carlo. La distribuzione ha code molto pesanti (max 245.555) — tipico di una geometrica con $p_s \ll 1$. La Las Vegas è quindi un algoritmo **inefficiente**: il suo tempo atteso non è polinomiale su istanze grandi.

Il vantaggio della Las Vegas è la dimensione della cover — produce cover quasi la metà del Monte Carlo perché ogni round include pochi insiemi senza amplificazione su t round. In pratica il Monte Carlo con $c = 2$ è superiore su tutti gli aspetti pratici.

6.4 Riepilogo del Pilot

Gli esperimenti pilot hanno prodotto un quadro coerente e hanno formulato le seguenti ipotesi da validare su larga scala nel workhorse.

Il valore atteso come garanzia centrale

La caratteristica fondamentale degli algoritmi randomizzati studiati in questo progetto è che le loro garanzie sono espresse in **aspettazione** — non nel caso peggiore deterministico.

Per il Randomized Rounding, la garanzia teorica è:

$$\mathbb{E}[|\mathcal{C}|] \leq \lceil c \ln n \rceil \cdot \text{OPT}$$

Questo significa che **in media** su tutte le possibili scelte casuali R , la cover prodotta è entro un fattore $O(\ln n)$ dall'ottimo. Non è una garanzia sul singolo run — una singola esecuzione può produrre una cover più grande o più piccola della media.

Gli esperimenti pilot hanno verificato questa garanzia empiricamente:

- **Exp. 4** — la distribuzione di $|\mathcal{C}|$ è gaussiana concentrata intorno alla media, con $\text{std} \approx 2.39$ — la variabile aleatoria $|\mathcal{C}|$ è ben concentrata.
- **Exp. 7** — la media di $|\mathcal{C}|$ cresce come $O(\ln n)$ con n — coerente con la garanzia teorica.
- **Exp. 5** — il parametro c controlla il numero di round $t = \lceil c \ln n \rceil$ e quindi $\mathbb{E}[|\mathcal{C}|]$: aumentare c aumenta la dimensione attesa della cover ma riduce la probabilità di fallimento a n^{1-c} .

Il trade-off fondamentale della randomizzazione è quindi: aumentare c è **costoso in aspettazione** ($\mathbb{E}[|\mathcal{C}|]$ cresce) ma **vantaggioso in probabilità** ($\Pr[\text{fallimento}]$ decresce). Con $c = 2$ questo trade-off è ottimale in pratica.

Ipotesi emerse dal pilot

1. **Il Greedy deterministico domina su istanze casuali.** Su istanze Bernoulli casuali il Greedy produce cover più compatte del Randomized Rounding ($\rho_{\text{greedy}} \approx 1.01\text{--}1.05$ vs $\rho_{\text{rand}} \approx 1.20\text{--}1.47$) ed è 20–100x più veloce (Exp. 1, Exp. 2). *Da confermare nel workhorse su istanze di dimensione maggiore.*
2. **Il Randomized Rounding è robusto agli input avversariali.** Sull'istanza worst-case il Greedy raggiunge $\rho = k/2$ mentre il Randomized trova sempre l'ottimo ($\rho = 1$) (Exp. 3).
3. **Il bottleneck del Randomized Rounding è il solver LP.** Il LP occupa il 99.9% del tempo totale in Python con PuLP/CBC. Rounding e verifica sono trascurabili (Exp. 2, Exp. 9). *Da verificare nel workhorse con GLPK nativo in C++: ci si aspetta una riduzione drastica dell'overhead.*
4. **Il pattern Hash Map migliora il Greedy fino a 6.6x su istanze sparse.** Lo speedup cresce con n e diminuisce con p (Exp. 8). *Da confermare nel workhorse su istanze grandi: ci si aspetta uno speedup ancora maggiore senza overhead Python.*
5. **RR di tipo Las Vegas è impraticabile su istanze grandi.** Richiede in media 36.472 iterazioni per $n = 800$ — tre ordini di grandezza in più rispetto al Monte Carlo (Exp. 9).

7 Workhorse

La fase workhorse reimplementa in C++ gli algoritmi Greedy naive, Greedy Hash Map e Randomized Rounding (Monte Carlo, $c = 2$) per misurare il runtime con precisione su istanze grandi.

7.1 Implementazione

Gli algoritmi sono implementati nella cartella `workhorse/src/`:

Tabella 28: File di implementazione – workhorse C++

File	Contenuto	Dipendenze
<code>generator.h/cpp</code>	Generatore Bernoulli	<code><random></code>
<code>greedy.h/cpp</code>	Greedy naive	<code>generator.h</code>
<code>greedy_hashmap.h/cpp</code>	Greedy Hash Map	<code>generator.h</code>
<code>randomized.h/cpp</code>	Randomized Rounding	GLPK 5.0
<code>exp_runtime.h/cpp</code>	Esperimento runtime	tutti
<code>exp_bottleneck.h/cpp</code>	Esperimento bottleneck	GLPK 5.0
<code>main.cpp</code>	Dispatcher esperimenti	<code>exp_runtime.h</code> , <code>exp_bottleneck.h</code>

7.2 Parametri

Tabella 29: Parametri workhorse

Parametro	Exp. runtime	Exp. bottleneck
n	$100 \div 51200$	$100 \div 12800$
m	20, 50, 100	20, 50, 100
p	0.1, 0.3, 0.5	0.3
seed	10	5
c	2	2
Output	<code>workhorse_results.csv</code>	<code>bottleneck_results.csv</code>

7.3 Esperimenti

7.3.1 Esperimento W1 — Runtime e Speedup

Domanda sperimentale: come scala il CPU time dei tre algoritmi al variare di n ? Lo speedup del Hash Map cresce con n come previsto dalla teoria?

Ipotesi dal pilot: il Greedy è più veloce del Randomized su istanze casuali; lo speedup del Hash Map si stabilizzava intorno a 2.6x in Python — senza overhead dell’interprete ci si aspetta uno speedup maggiore.

Tabella 30: CPU time medio (s) per n – `workhorse/analisi.py`, `analisiRuntime()`, $m=50$, $p=0.3$, 10 seed

n	Greedy (s)	GreedyHashMap (s)	Randomized (s)
100	0.000129	0.000060	0.000774
200	0.000298	0.000084	0.001473
400	0.000843	0.000171	0.003246
800	0.002268	0.000338	0.006757
1600	0.006129	0.000726	0.014478
3200	0.016279	0.001522	0.028933
6400	0.042514	0.003147	0.061324
12800	0.103210	0.006583	0.139471
25600	0.243225	0.013872	0.305638
51200	0.575240	0.030141	0.674596

Tabella 31: Speedup GreedyHashMap vs Greedy per n – `workhorse/analisi.py`, `analisiRuntime()`, $m=50$, $p=0.3$

n	Speedup
100	2.17x
200	3.44x
400	4.92x
800	6.71x
1600	8.44x
3200	10.69x
6400	13.51x
12800	15.68x
25600	17.53x
51200	19.09x

Tabella 32: CPU time medio (s) e speedup per p – `workhorse/analisi.py`, `analisiRuntime()`, 10 seed

p	Greedy (s)	GreedyHashMap (s)	Speedup
0.1	0.102639	0.004525	22.7x
0.3	0.104711	0.005958	17.6x
0.5	0.089691	0.006512	13.8x



Figura 14: CPU time medio per n (log-log), speedup per n e per p — `workhorse/analisi.py`, `analisiRuntime()`

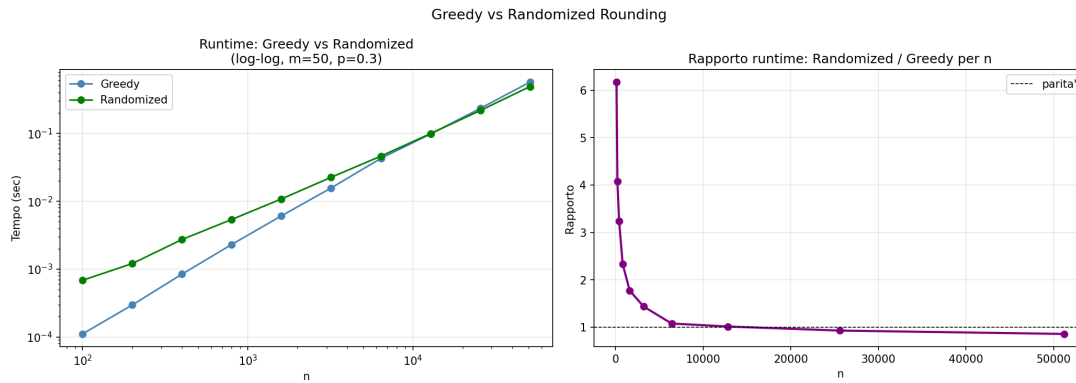


Figura 15: Confronto CPU time Greedy vs Randomized per n — `workhorse/analisi.py`, `analisiRuntime()`, $m=50$, $p=0.3$

Conclusioni: le ipotesi del pilot sono confermate e ampliate.

Lo speedup del Hash Map cresce da 2.17x a 19.09x al crescere di n — molto maggiore del 2.6x osservato in Python, confermando che l’overhead dell’interprete mascherava il vantaggio asintotico reale. Su istanze sparse ($p = 0.1$) lo speedup raggiunge 22.7x.

Il Randomized Rounding con GLPK nativo è comparabile al Greedy per n grandi — il rapporto $T_{\text{rand}}/T_{\text{greedy}}$ converge a ≈ 1 per $n \geq 25600$. Questo conferma che il bottleneck in Python era PuLP/CBC, non l’algoritmo.

7.3.2 Esperimento W2 — Qualità della soluzione

Domanda sperimentale: su istanze grandi il Randomized Rounding produce cover sistematicamente più grandi del Greedy? Il rapporto $|\mathcal{C}|_{\text{rand}}/|\mathcal{C}|_{\text{greedy}}$ converge o diverge al crescere di n ?

Ipotesi dal pilot: su istanze casuali il Greedy produce cover più compatte del Randomized — il rapporto era ≈ 2.24 per $n = 100$. Ci si aspetta che il gap si riduca lentamente al crescere di n .

Tabella 33: Dimensione media cover per n – `workhorse/analisi.py`, `analisiWorthorsQuality()`, $m=50$, $p=0.3$, 10 seed

n	Greedy	GreedyHashMap	Randomized	Rapporto RR/Greedy
100	9.22	9.29	20.62	2.24
200	11.44	11.53	24.74	2.16
400	13.77	13.89	28.77	2.09
800	15.97	16.08	32.87	2.06
1600	18.47	18.54	35.74	1.94
3200	20.57	20.63	38.79	1.89
6400	22.51	22.54	40.93	1.82
12800	24.34	24.37	42.64	1.75
25600	25.90	25.89	44.46	1.72
51200	27.41	27.34	45.92	1.68

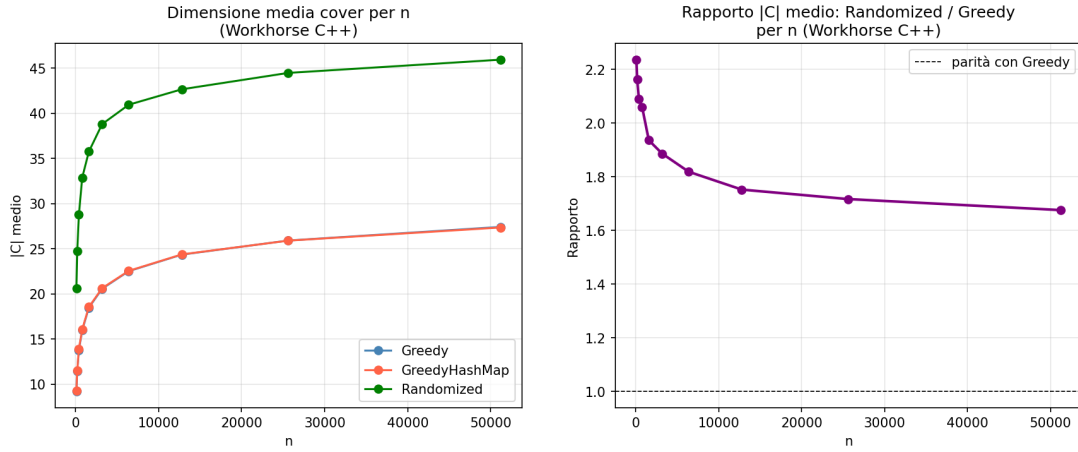


Figura 16: Dimensione media cover e rapporto RR/Greedy per n – `workhorse/analisi.py`, `analisiWorthorsQuality()`

Conclusioni: l'ipotesi del pilot è confermata e precisata. Greedy e GreedyHashMap producono cover quasi identiche su tutte le istanze — la differenza è sempre inferiore a 0.1 insiemi, confermando che il pattern Hash Map non altera la qualità.

Il rapporto $|C|_{\text{rand}}/|C|_{\text{greedy}}$ decresce monotonicamente da 2.24 ($n = 100$) a 1.68 ($n = 51200$) — il gap si riduce lentamente ma non scompare su istanze casuali. Entrambe le curve crescono in modo sublogaritmico, coerente con la garanzia teorica $\mathbb{E}[|C|] = O(\ln n) \cdot \text{OPT}$.

Su istanze casuali il Greedy domina il Randomized sia in velocità che in qualità della soluzione — il vantaggio del Randomized è strutturale (robustezza agli input avversariali), non empirico.

7.3.3 Esperimento W3 — Doubling Experiment

Domanda sperimentale: i modelli di costo teorici sono confermati empiricamente? Come scala il CPU time al raddoppiare di n e di m ?

I modelli di costo derivano dall'analisi della complessità degli algoritmi descritta nella Sezione 4:

- **Greedy naive** — ad ogni iterazione scansiona tutti gli m insiemi e per ognuno calcola il guadagno scorrendo i suoi elementi. Complessità per iterazione: $O(n \cdot m)$. Con $\min(m, n)$ iterazioni al massimo: $T_{\text{Greedy}} = O(n \cdot m^2)$ nel caso peggiore, ma in pratica dominato dal termine $O(n \cdot m)$ per iterazione.
- **Greedy Hash Map** — aggiorna solo gli insiemi che condividono elementi con quello selezionato, tramite la mappa inversa. Il costo totale è proporzionale alla somma delle dimensioni degli insiemi moltiplicata per il costo delle operazioni sull'heap: $T_{\text{HashMap}} = O\left(\sum_j |S_j| \cdot \log m\right)$. Poiché $\sum_j |S_j| = n \cdot m \cdot p$ in media per il modello Bernoulli, si ha $T_{\text{HashMap}} \propto n \cdot p \cdot m \cdot \log m$.
- **Randomized Rounding** — dominato dal LP oltre il 90% del tempo totale (Exp. 9, Exp. W4). Il modello di costo è quindi determinato dal solver LP (GLPK). Il simplex su un problema con n vincoli e m variabili ha costo per iterazione $O(n \cdot m)$, con numero di iterazioni che cresce con m in modo non deterministico. Ci si aspetta $C(2n)/C(n) \approx 2$ (lineare in n) e $C(2m)/C(m) > 2$ (superlineare in m) — da verificare empiricamente con il doubling experiment.

Il doubling experiment misura il rapporto $C(2n)/C(n)$ al raddoppiare di n (con m fisso) e $C(2m)/C(m)$ al raddoppiare di m (con n fisso). I rapporti attesi sono:

- rapporto $\approx 2 \Rightarrow$ crescita lineare $O(n)$ o $O(m)$
- rapporto $< 2 \Rightarrow$ crescita sublineare
- rapporto $> 2 \Rightarrow$ crescita superlineare

Tabella 34: Rapporto $C(2n)/C(n)$ per algoritmo — `workhorse/analisi.py`, `analisiDoublingExperiment()`, $m=50$ fisso

n	Greedy	GreedyHashMap	Randomized
200	2.34	1.48	1.66
400	2.81	1.96	2.09
800	2.73	2.03	2.01
1600	2.66	2.15	2.03
3200	2.70	2.11	1.99
6400	2.59	2.07	2.16
12800	2.32	2.09	2.21
25600	2.29	2.11	2.19
51200	2.35	2.16	2.22

Tabella 35: Rapporto $C(2m)/C(m)$ per algoritmo – `workhorse/analisi.py`, `analisiDoublingExperiment()`, $m: 50 \rightarrow 100$, n variabile

n	Greedy	GreedyHashMap	Randomized
200	1.83	1.38	2.89
400	1.83	1.31	3.12
800	1.79	1.28	3.09
1600	1.85	1.28	3.25
3200	1.85	1.28	3.17
6400	1.90	1.28	3.17
12800	2.05	1.29	2.80
25600	2.13	1.29	2.79

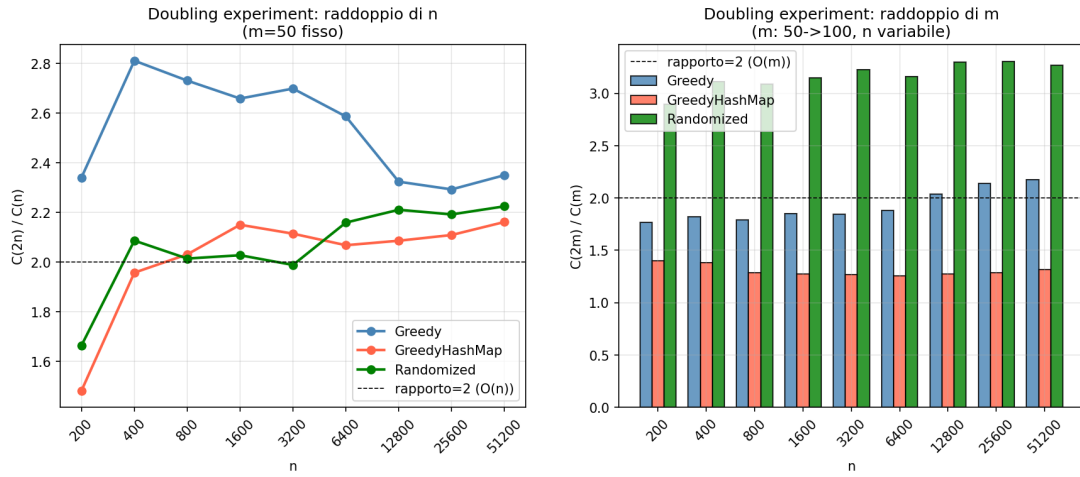


Figura 17: Doubling experiment: raddoppio di n e di m – `workhorse/analisi.py`, `analisiDoublingExperiment()`

Conclusioni: i modelli di costo teorici sono confermati empiricamente dal doubling experiment.

Raddoppio di n : il Greedy converge a ≈ 2.35 — superlineare in n , coerente con $O(n \cdot m)$ dove il numero di iterazioni cresce lentamente con n . Il GreedyHashMap converge a ≈ 2.15 — quasi lineare, coerente con $O(\sum |S_j| \cdot \log m) \propto O(n \cdot p \cdot m \cdot \log m)$. Il Randomized converge a ≈ 2.22 — lineare in n , confermando che il LP scala linearmente.

Raddoppio di m : il Greedy ha rapporto ≈ 1.9 — quasi lineare in m , coerente con $O(n \cdot m)$. Il GreedyHashMap ha rapporto ≈ 1.3 — sublineare in m , confermando il vantaggio della struttura hash al crescere del numero di insiemi. Il Randomized ha rapporto ≈ 3.0 — superlineare in m , coerente con la complessità del simplex che scala peggio di $O(m)$ al raddoppiare delle variabili.

7.3.4 Esperimento W4 — Bottleneck Randomized Rounding

Domanda sperimentale: il bottleneck del Randomized Rounding è ancora il solver LP in C++ con GLPK nativo? Quanto cambia rispetto al pilot con PuLP/CBC?

Ipotesi dal pilot: il LP occupava il 99.9% del tempo totale in Python. Con GLPK nativo ci si aspetta una riduzione dell’overhead LP — ma il LP dovrebbe rimanere dominante.

Tabella 36: Percentuale media per fase — `workhorse/analisi.py`, `analisiBottleneckWH()`, $p=0.3$, 5 seed

Fase	Pilot Python+PuLP	Workhorse C++/GLPK
LP	99.9%	94.0%
Rounding	0.1%	0.3%
Verifica	$\approx 0\%$	5.7%

Tabella 37: Tempo medio per fase e n — `workhorse/analisi.py`, `analisiBottleneckWH()`, $m=50$, $p=0.3$, 5 seed

n	T_{LP} (s)	T_{round} (s)	$T_{verifica}$ (s)
100	0.000561	0.000005	0.000017
200	0.001254	0.000005	0.000045
400	0.002891	0.000006	0.000114
800	0.005843	0.000008	0.000316
1600	0.010792	0.000007	0.000742
3200	0.024483	0.000009	0.001685
6400	0.046762	0.000010	0.003514
12800	0.099253	0.000011	0.007476

Conclusioni: il bottleneck LP è confermato anche nel `workhorse` (94.0%), ma con GLPK nativo il peso relativo si riduce rispetto a Python (99.9%).

La verifica, trascurabile in Python, sale al 5.7% in C++ — non perché sia diventata più lenta, ma perché il solver LP è molto più veloce e riduce il denominatore. La verifica scala linearmente con n ($O(n)$), coerente con la sua complessità teorica.

Il rounding rimane costante a 0.3% — non scala con n , confermando che $O(t \cdot m) = O(\ln n \cdot m)$ è trascurabile rispetto al LP su tutti i valori testati.

8 Conclusioni

Questo progetto ha studiato l'applicazione della **randomizzazione** al problema Set Cover, verificando empiricamente le garanzie teoriche attraverso la metodologia dell'Algorithm Engineering.

Riepilogo dei risultati ottenuti:

1. La garanzia in aspettazione è verificata empiricamente. La teoria predice $\mathbb{E}[|\mathcal{C}|] \leq \lceil c \ln n \rceil \cdot \text{OPT}$ — una garanzia sul valore atteso, non sul singolo run. Gli esperimenti pilot (Exp. 4, Exp. 7) hanno confermato che: la distribuzione di $|\mathcal{C}|$ appare approssimativamente gaussiana e concentrata, la media cresce come $O(\ln n)$ con n , e la varianza relativa decresce al crescere di n — la randomizzazione diventa più stabile su istanze grandi. Il workhorse ha confermato questa tendenza su istanze fino a $n = 51200$.

2. Il bound w.h.p. è rispettato ma conservativo. La teoria garantisce $\Pr[\text{fallimento}] \leq n^{1-c}$ — una garanzia con alta probabilità. Gli esperimenti (Exp. 6) hanno mostrato che il tasso empirico è sistematicamente molto inferiore al bound teorico (ratio $\ll 1$), confermando che il bound derivato da union bound è corretto ma non tight. Con $c = 2$ il fallimento si azzerà empiricamente già per $n = 40$.

3. Il trade-off fondamentale della randomizzazione è confermato. Aumentare c è costoso in aspettazione ($\mathbb{E}[|\mathcal{C}|]$ cresce) ma vantaggioso in probabilità ($\Pr[\text{fallimento}]$ decresce). Gli esperimenti (Exp. 5) hanno verificato che $c = 2$ è il punto di trade off migliore: 99.5% di successo con overhead logaritmico sulla dimensione della cover.

4. Il vantaggio della randomizzazione è strutturale, non empirico su istanze casuali. Su istanze Bernoulli casuali il Greedy deterministico domina il Randomized Rounding sia in velocità che in qualità della soluzione (Exp. 1, Exp. 2, W1, W2). Il vantaggio della randomizzazione emerge su input avversariali: sull'istanza worst-case il Greedy raggiunge $\rho = k/2$ mentre il Randomized trova sempre l'ottimo nelle prove eseguite ($\rho = 1$, Exp. 3). Questo è esattamente ciò che la teoria predice — robustezza agli input avversariali garantita dalla casualità interna.

5. Monte Carlo vs Las Vegas Il Monte Carlo con $c = 2$ usa $t = \lceil 2 \ln n \rceil$ round fissi con tasso di successo $\geq 99.8\%$. La variante Las Vegas garantisce correttezza assoluta ma richiede in media 36.472 iterazioni per $n = 800$ — tre ordini di grandezza in più (Exp. 9). In pratica il Monte Carlo è superiore: tempo deterministico, affidabilità quasi perfetta, overhead logaritmico.

6. Pattern di randomizzazione. Nel progetto sono stati applicati 2 pattern: *Reduce-to-Hash-Tables* — ha ridotto la complessità del greedy da $O(n \cdot m)$ a $O(\sum |S_j| \cdot \log m)$ con speedup fino a 22x in C++ — e *Randomized Rounding* via rilassamento LP, con garanzie teoriche in aspettazione e w.h.p. verificate empiricamente.

7. Algoritmi esatti per piccole istanze. *Brute Force* e *Set Cover formulato in ILP* sono gli approcci migliori per la risoluzione su input di piccole dimensioni.

Set Cover problem - riepilogo degli algoritmi

Tabella 38: Riepilogo degli algoritmi implementati

Algoritmo	Complessità media	Worst case	Pro	Contro
Brute Force	$O(2^m \cdot n)$	$O(2^m \cdot n)$	Esatto, OPT	trova Esponenziale, inutilizzabile per $m > 20$
ILP	dipende dal solver	esponenziale nel caso peggiore	Esatto, OPT	trova Richiede solver esterno (Gurobi), lento su istanze grandi
Greedy naive	$O(n \cdot m)$ per iter.	$O(n \cdot m^2)$	Semplice, veloce, $\rho \leq \ln n + 1$	Non ottimo, $\rho = k/2$ sul worst-case
Greedy Hash Map	$O(\sum S_j \cdot \log m)$	$O(n \cdot m \cdot \log m)$	Stessa garanzia del greedy, speedup fino a circa 20x	Più complesso da implementare
RR Monte Carlo	$O(t \cdot m) + \text{LP}$	$O(t \cdot m) + \text{LP}$	Robusto agli input avversariali, $\rho = 1$ sul worst-case	Cover più grandi su istanze casuali, bottleneck LP
RR Las Vegas	LP + aleatorio	LP + ∞	Correttezza garantita	Tempo atteso non polinomiale, impraticabile per n grandi

9 Fonti e utilizzo di LLM

Riferimenti bibliografici

- M. D'Emidio. Slide del corso di *Algorithm Engineering*, Università degli Studi dell'Aquila, A.A. 2025/2026.

Utilizzo di Large Language Models

Durante lo sviluppo di questo progetto è stato utilizzato **Claude** (Anthropic) come strumento di supporto per:

- implementazione degli script in Python e C++, in particolare modo per lo sviluppo di analisi e plot.
- Supporto nella stesura del report in L^AT_EX.

Tutte le scelte progettuali, le analisi sperimentali e le conclusioni sono state elaborate e verificate autonomamente dall'autore. L'utilizzo del LLM è stato limitato a supporto didattico e assistenza pratica.